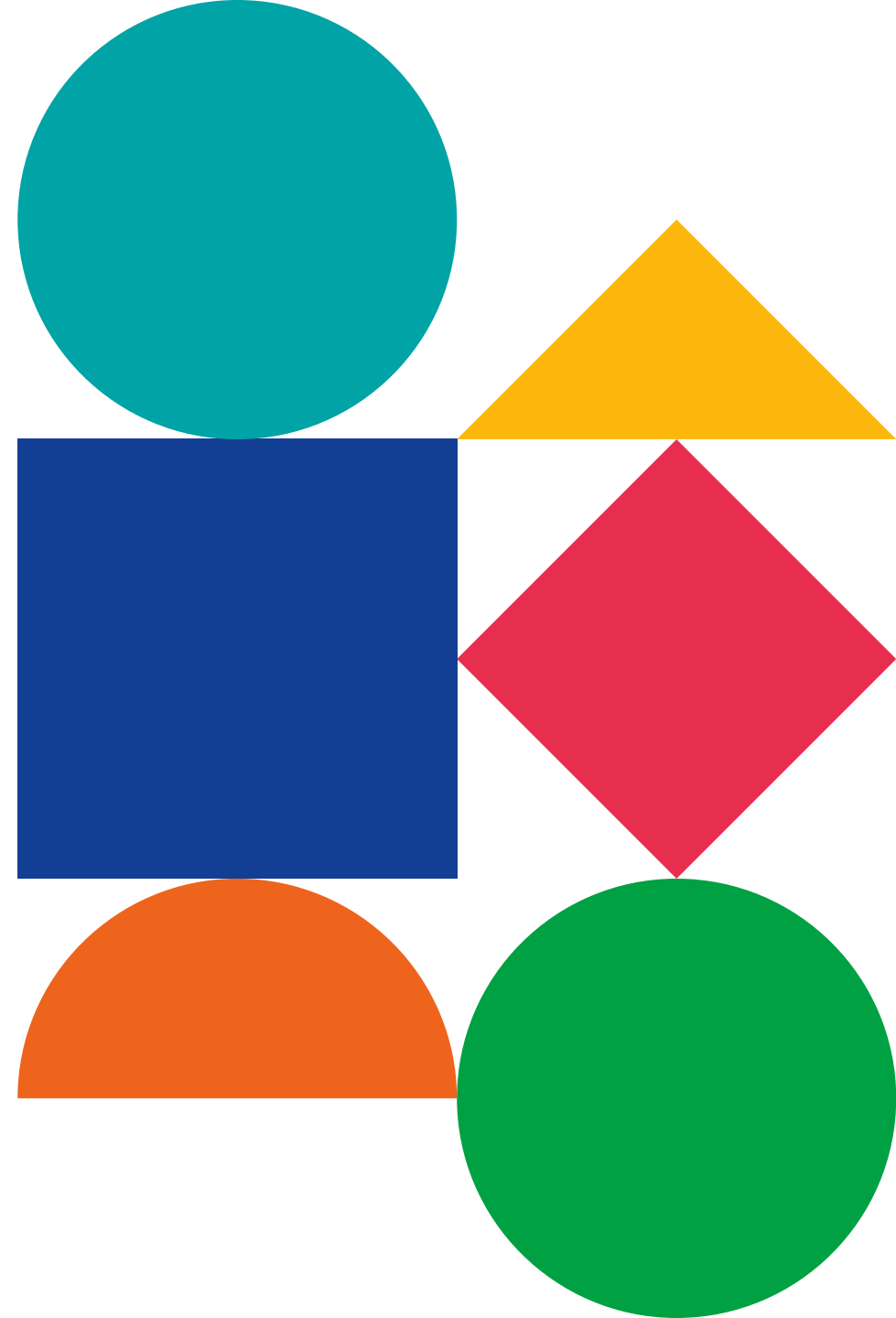


Versioning with Git & GitHub

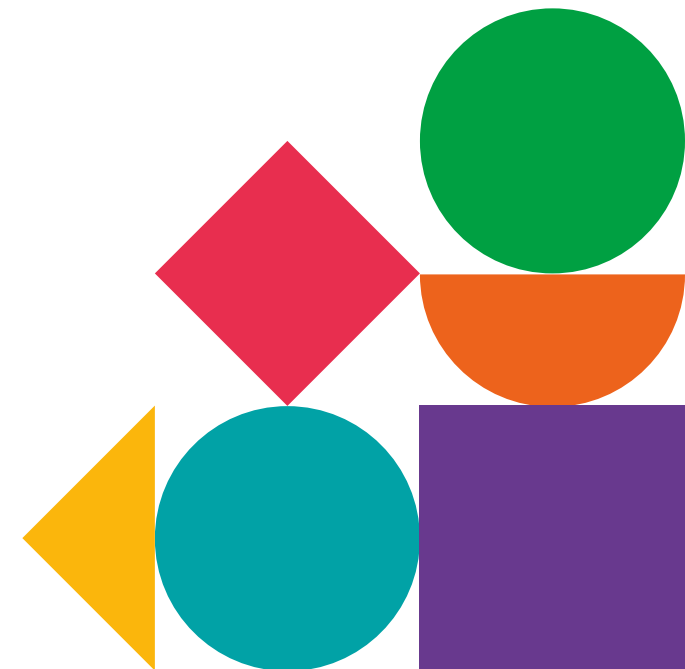
For code and documentation - Day 2

Bruna Piereck

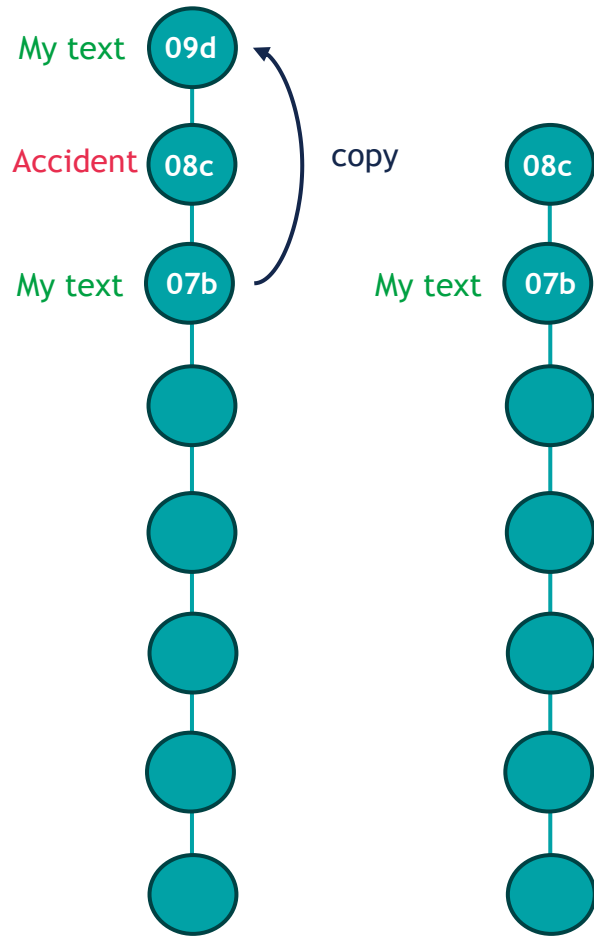
Dec 2025



Review



Restore a previous good state

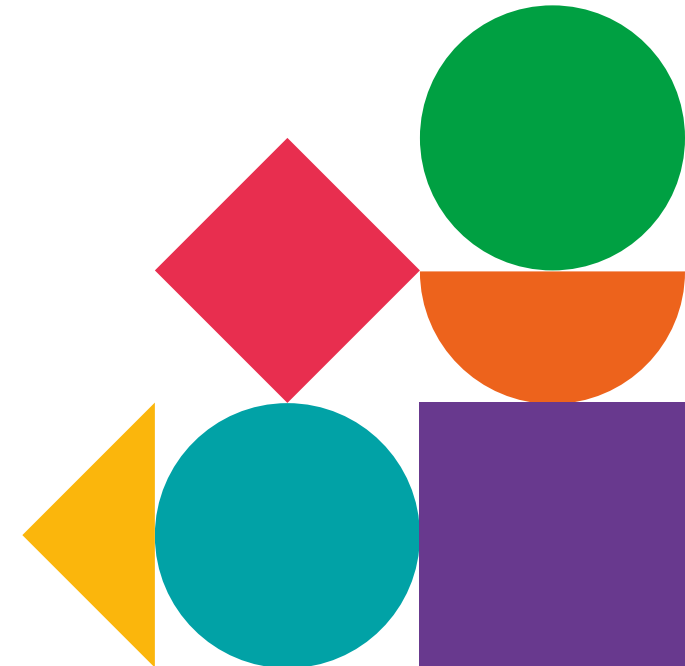


\$ git revert

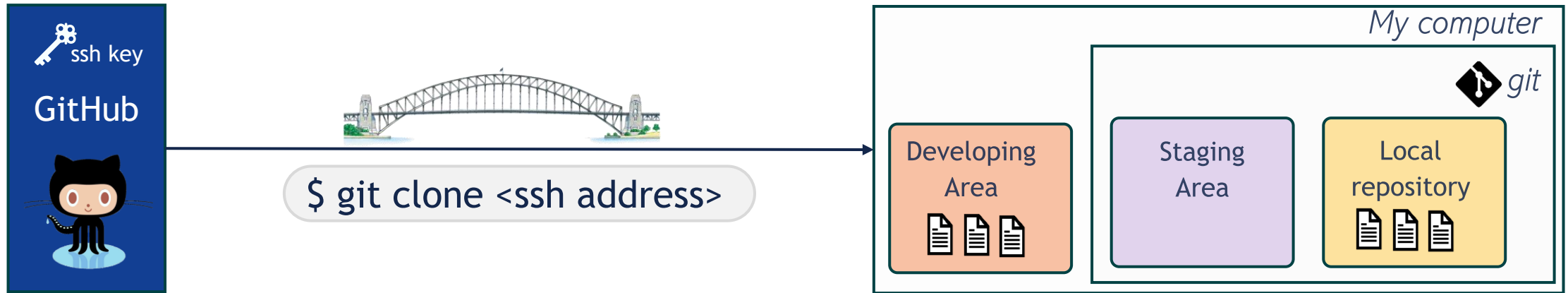
- Copy back the “good” commit
- Keeps in the history
 - committed
 - Reverted

\$ git reset

- Dangerous!!
- Discard a commit
- “Unstage” file



From remote to local

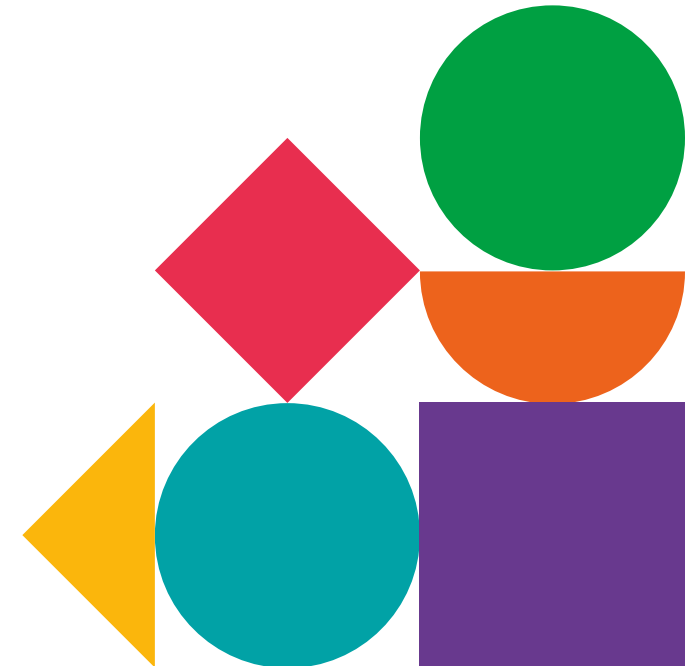


- Downloads repository content (creates dev. area)
- Adds the bridge immediately
- Creates a local repository automatically
- Only works if you have set a ssh key

ACTIVITY

A. Check if your notes have all you need

- Concepts:
 - Git vs GitHub
 - Conceptual areas
 - Meaningful message git commit (Why, How, Effects, limitations)
 - Staging area (Why and how to use)
 - README files usage
 - .gitignore file usage

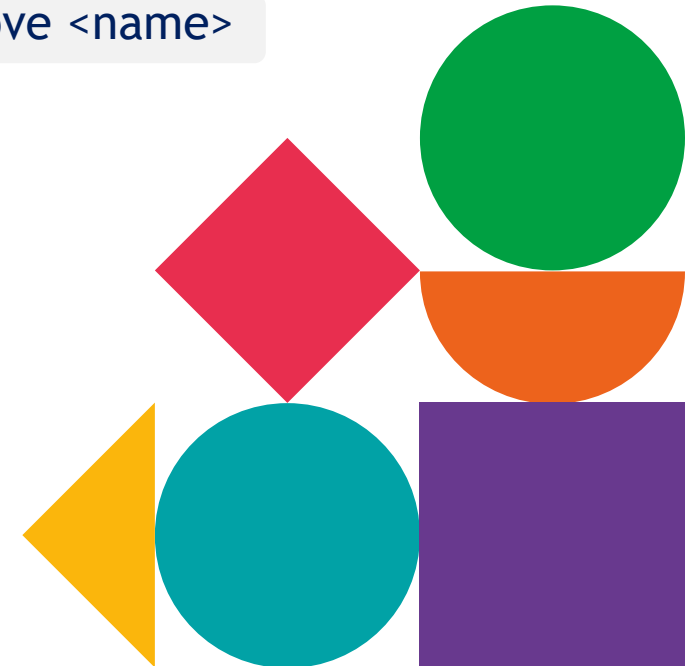


ACTIVITY

B. Check if your notes have all you need

■ Commands:

- Basic routine
 - `git status, git add, git commit, git push`
- Sync remote to local
 - `git pull`
- From local to remote
 - `git remote add origin <ssh address>`
- From remote to local
 - `git clone`
- Comparing commits
 - `git diff & git show`
- Check the history
 - `git log`
- Recover to a previous state
 - `git reset, git revert`
- Rewrite the last commit message
 - `git commit -amend`
- Delete the “bridge”
 - `git remote (to list the name - origin)`
 - `git remote remove <name>`



REVIEW QUIZ

 **Kahoot!**





Experimenting Risk Free

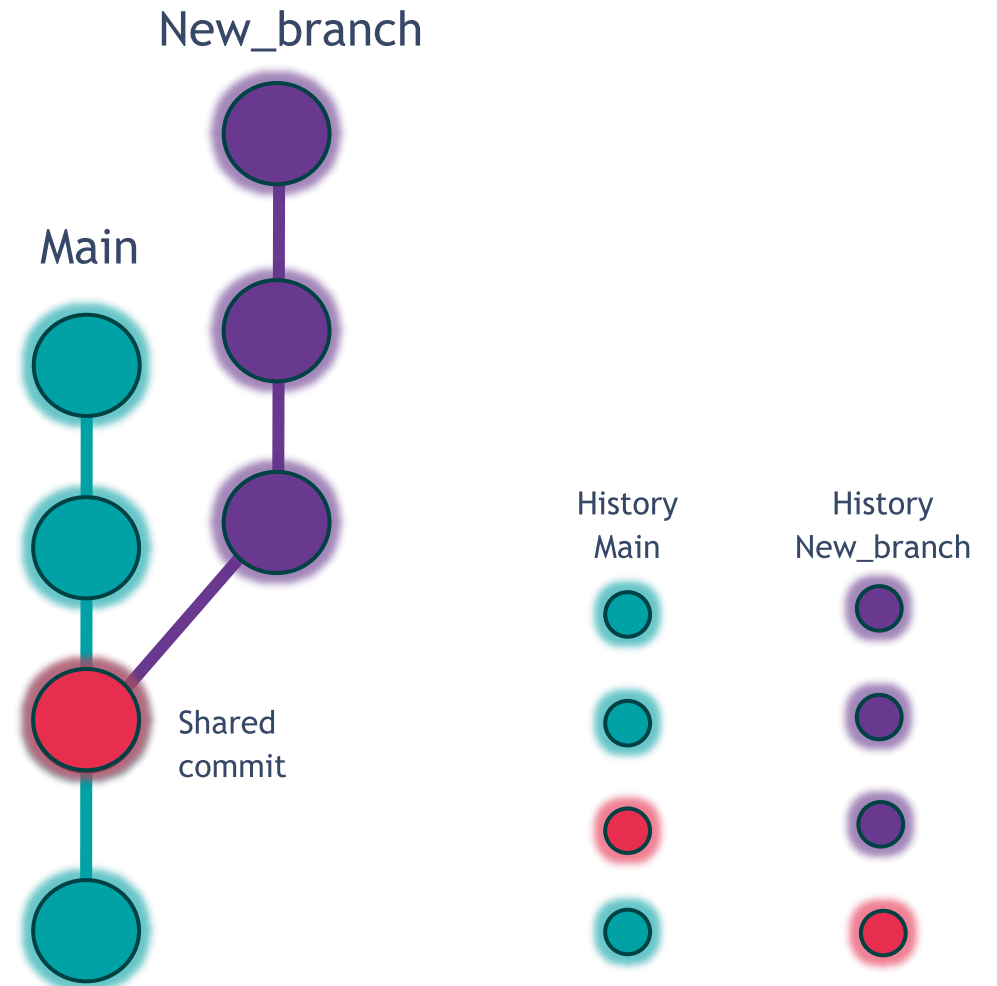
Alternative
timeline

Alternative history

```
user$ git branch <name>
```

Branch

- Has a specific name
- Alternative history of commits
 - Independent timeline
- Shared commit between timelines



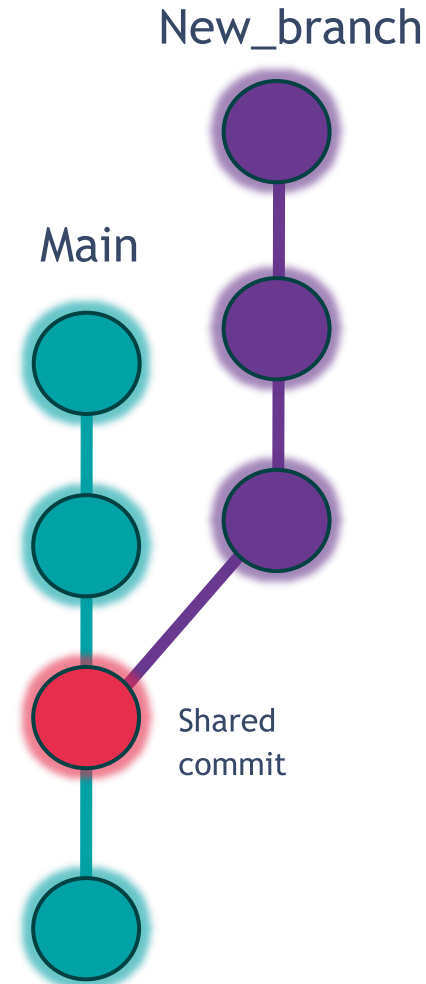
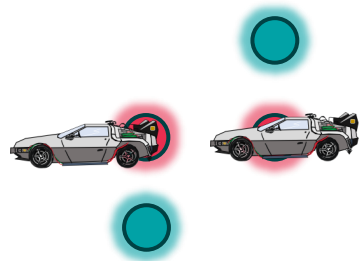
Alternative history

```
user$ git branch <name>
```

Utility

1. Experiment risk free
2. Collaborating in a project
3. Work in different tasks
4. Testing different possible solutions

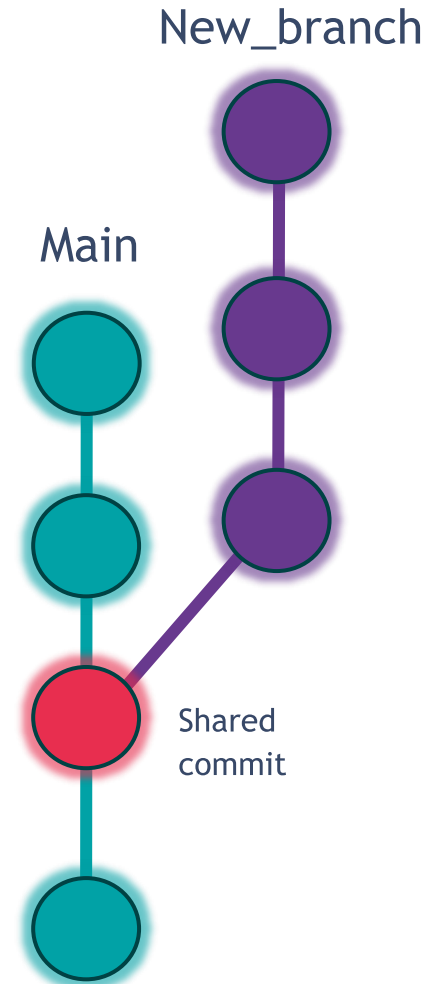
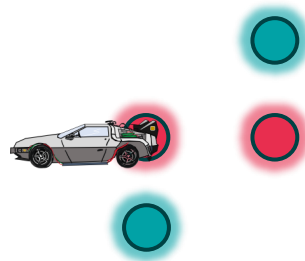
P.s.: Creating $\neq$ Working on



Moving between timelines

```
user$ git checkout <branch_name>
```

- Move between branches
- Move between commits
(!! Dangerous)

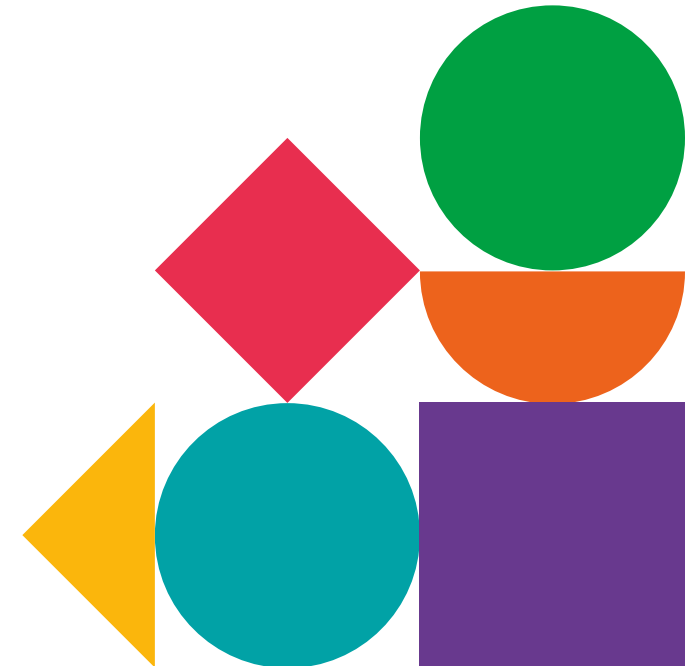


ACTIVITY

1. Making use of branches

- Go to your project
 - Sync all the content
 - Create your branch
`git branch branches_concept`
 - Checkout to your new branch
`git checkout branches_concept`
 - Write the concepts of Branching & Checkout
 - Write why branching can be useful
 - Commit (think of your message) and Push

- Why was changed
- How this addresses the issue
- Effects due the change
- Limitations of the change

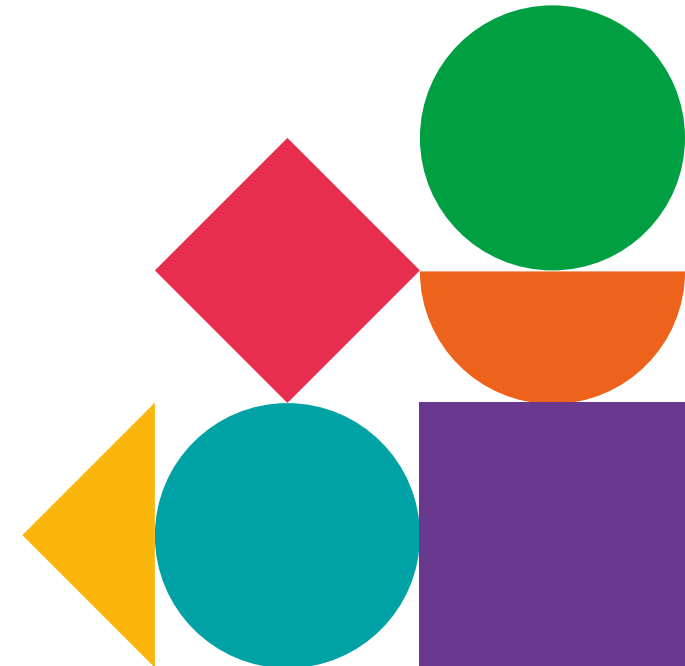


ACTIVITY

2. Making use of branches (Part 2)

- Checkout back to Main/Master
 - Create a new branch
`git branch new_command`
 - Checkout to your new branch
`git checkout new_command`
 - Write the commands
 - How to create a branch
 - How to move between branches
 - Commit (think of your message) and Push

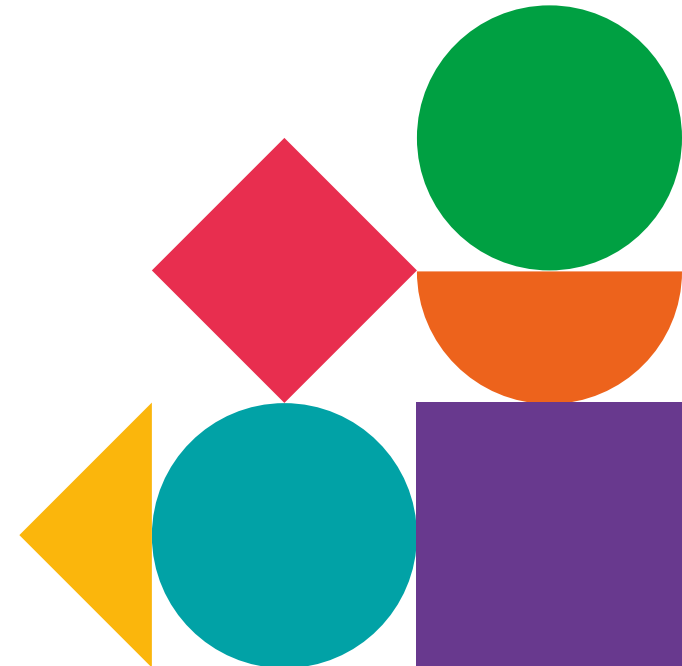
- Why was changed
- How this addresses the issue
- Effects due the change
- Limitations of the change



New branches

```
user$ git branch -- list
```

- Check local branches

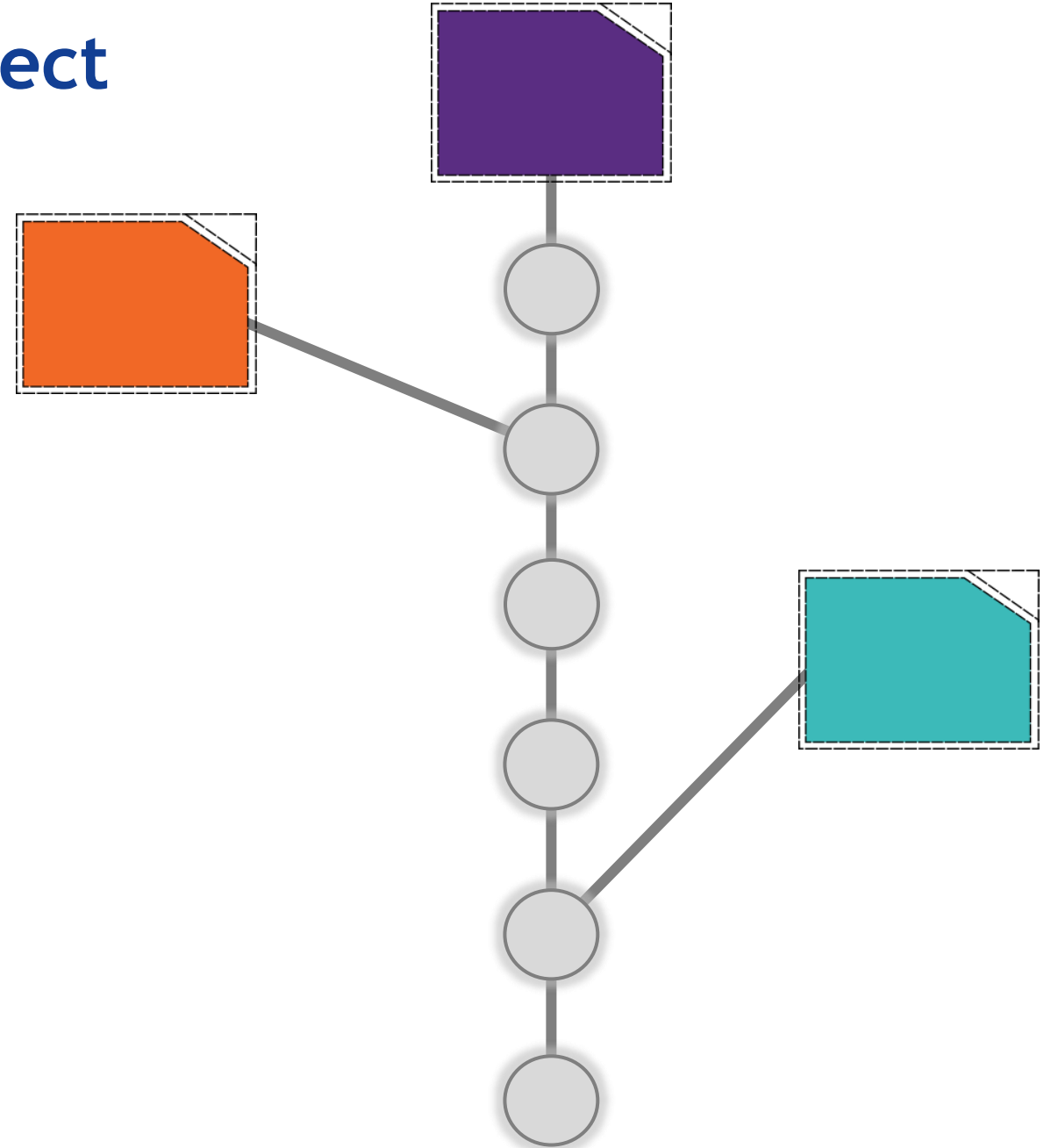
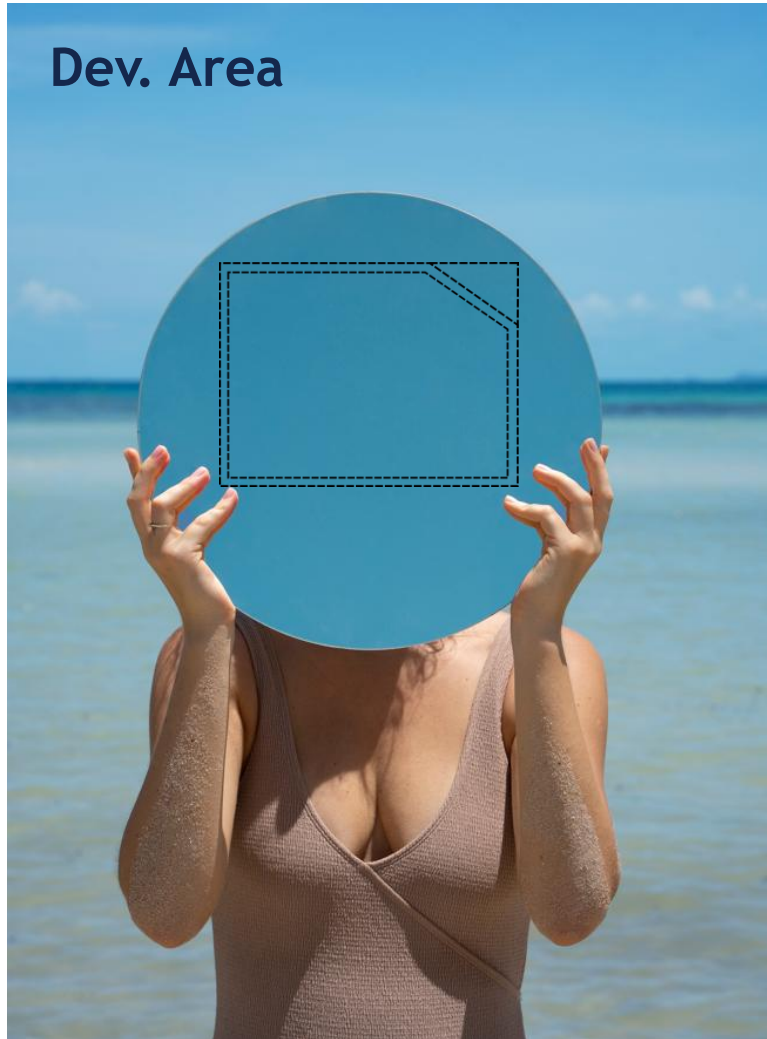


The “Mirror” effect

Dynamic
Dev. area

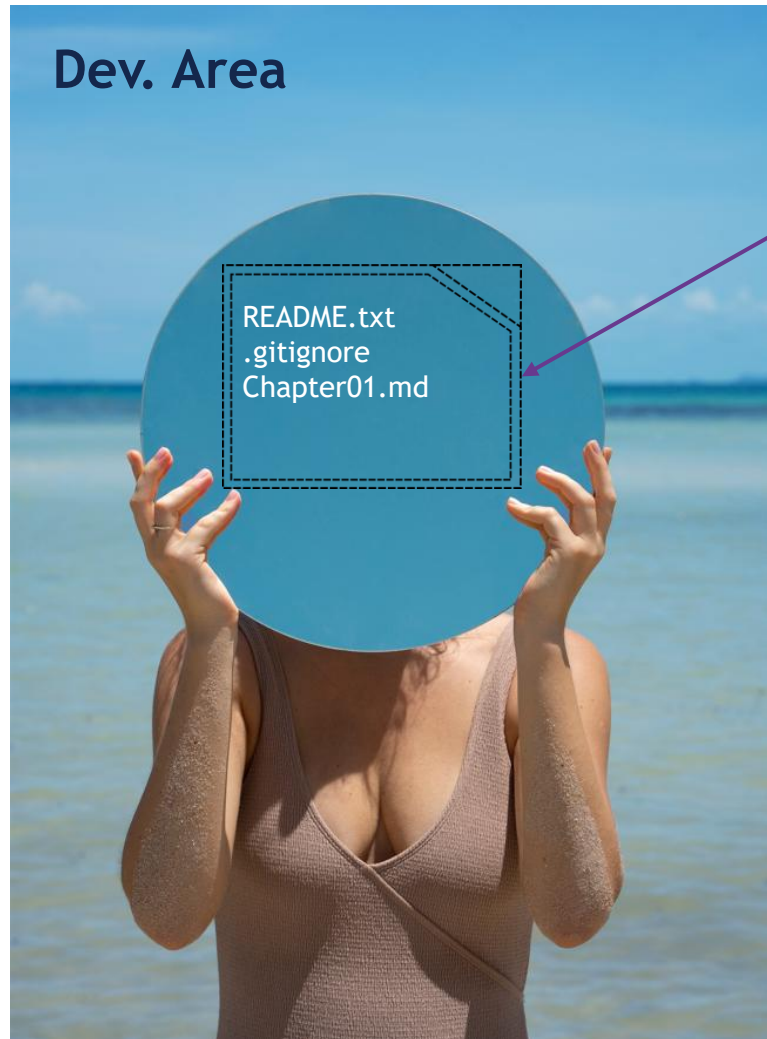


Branches: The “mirror” effect

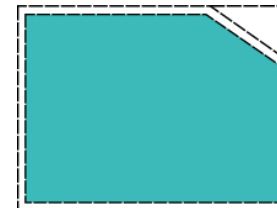
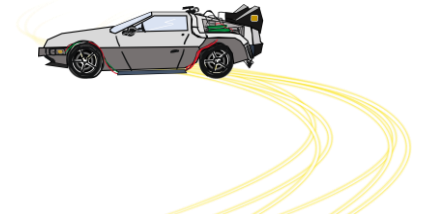


Branches: The “mirror” effect

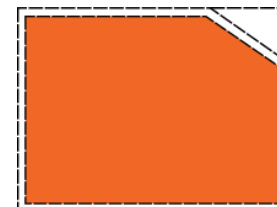
Git checkout vib_green



README.txt
.gitignore
Chapter01.md



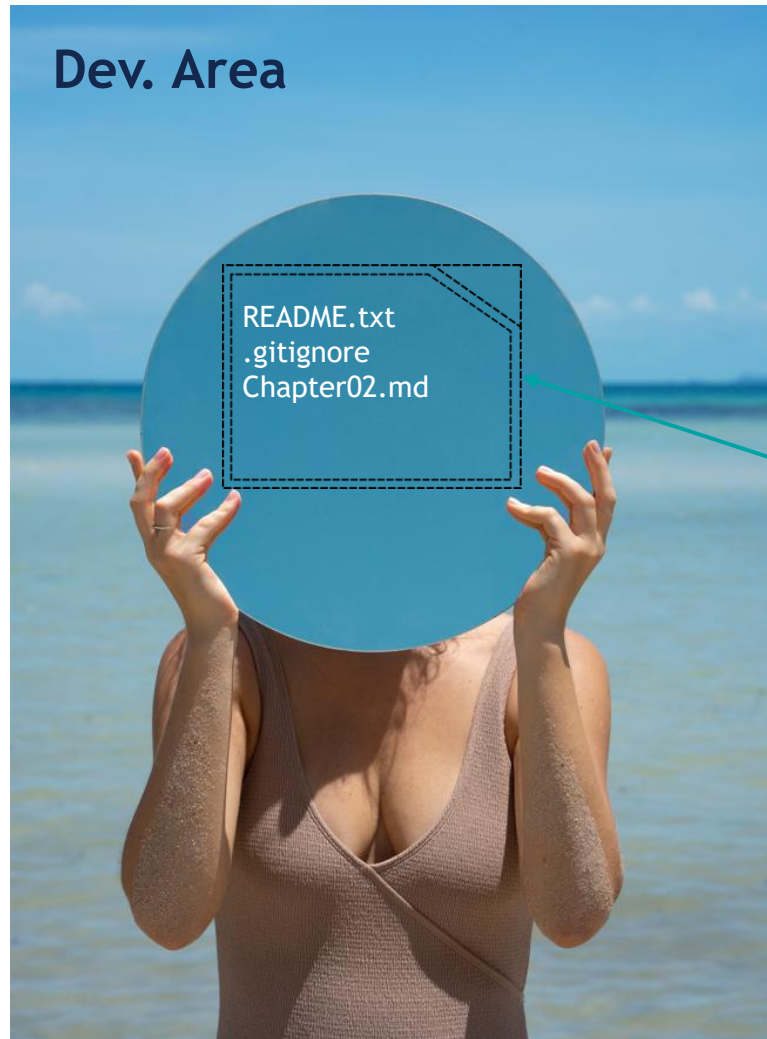
README.txt
.gitignore
Chapter02.md



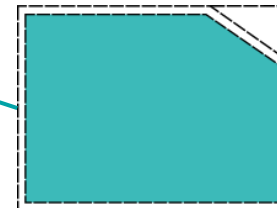
README.txt
.gitignore
Chapter01.md
Illustrations

Branches: The “mirror” effect

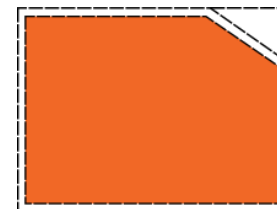
Git checkout vib_green



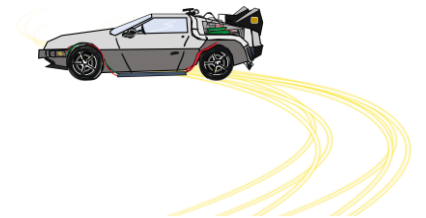
README.txt
.gitignore
Chapter01.md



README.txt
.gitignore
Chapter02.md



README.txt
.gitignore
Chapter01.md
Illustrations

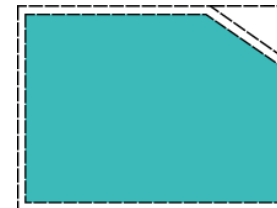


Branches: The “mirror” effect

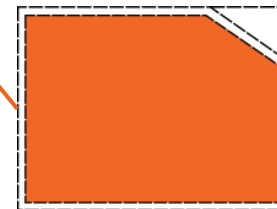
Git checkout vib_green



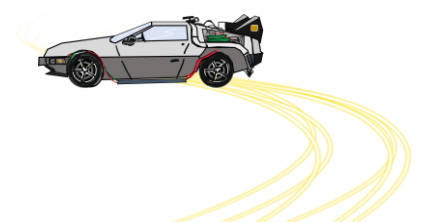
README.txt
.gitignore
Chapter01.md



README.txt
.gitignore
Chapter02.md



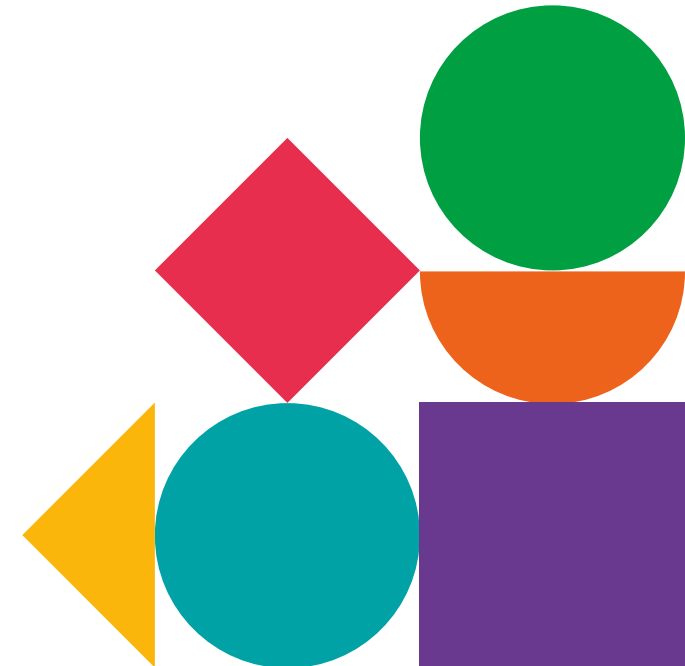
README.txt
.gitignore
Chapter01.md
Illustrations



ACTIVITY

3. Compare content in both timelines

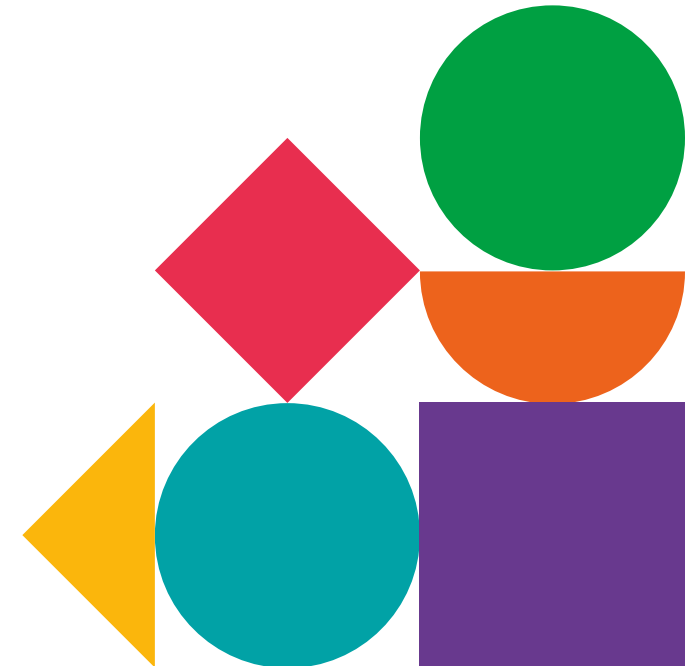
- Check your branch history
- Close all files
- Checkout to the main branch
`git checkout main`
- Check the history
 - Is there any differences?
- Open the files
 - Is there any information missing?
 - Why?



ACTIVITY

4. Working with branches

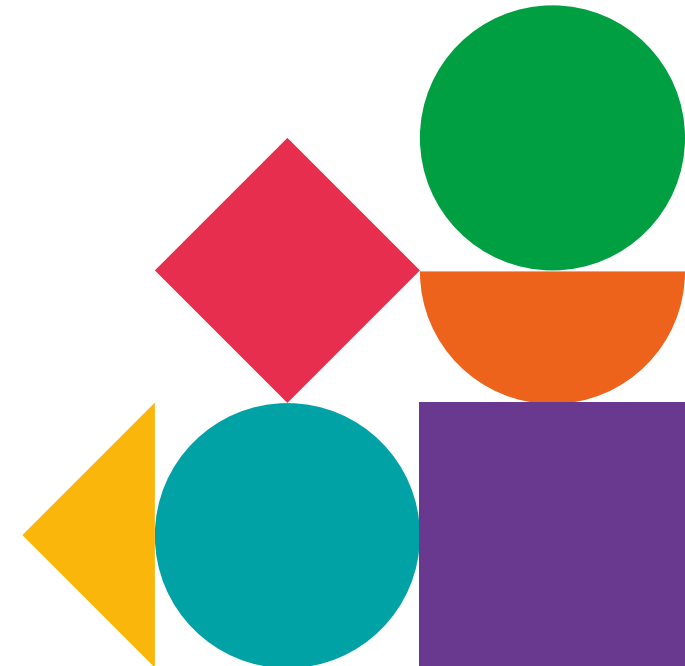
- Create a new_branch
 - `git branch mirror_effect`
 - `git checkout mirror_effect`
- Write about the “mirror” effect
- Commit & Push



ACTIVITY

5. Comparing

- Close all files
- Move back to main
 - Is the content there?
 - Why?
- Can you see all branches in GitHub?



Collaborating



GitHub brings
you together

GitHub interface showing repository settings for `name_of_myRepo`.

The **Settings** tab is selected in the top navigation bar.

The **General** section is active in the left sidebar.

The **Collaborators** link is highlighted in the left sidebar.

The **Repository name** field shows `name_of_myRepo` with a **Rename** button.

The **Releases** section is visible below the General section.

Chose what repository you want

Find the repository settings

In access management click Collaborators

GitHub will ask some kind of authentication

Confirm access

Signed in as `@bpiereck`

xxxxxx

Verify

Having problems?

- Send a code via email

Tip: You are entering `sudo mode`. After you've performed a sudo-protected action, you'll only be asked to re-authenticate again after a few hours of inactivity.

Collaborate to specific repositories

3

Check your mailbox
& accept the collaboration



bpiereck
Bruna Piereck Moura
Bruna.piereckmoura@vib.be

Add people to name_of_myRepo

Search by username, full name, or email

Find people

Cancel

Add to repository

bpiereck / name_of_myRepo

Code

Issues

Pull requests

Actions

Projects

Wiki

Security

Insights

Settings

General

Access

Collaborators

Moderation options

Code and automation

Rules

Actions

Models

Webhooks

Copilot

Environments

Collaborators and teams

Public repository

This repository is public and visible to anyone

Manage visibility

Direct access

0 collaborators have access to this repository. Only you can contribute to this repository.

Manage access

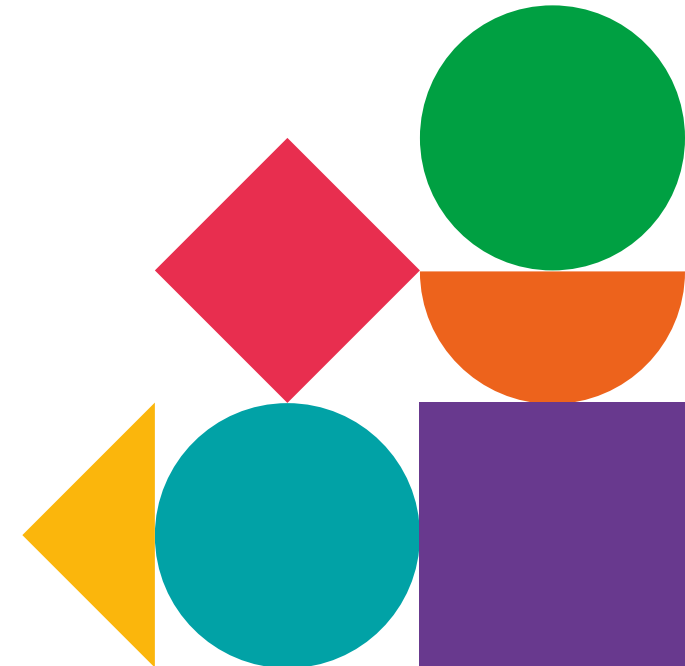
You haven't invited any collaborators yet

Add people

ACTIVITY

6. We will work together

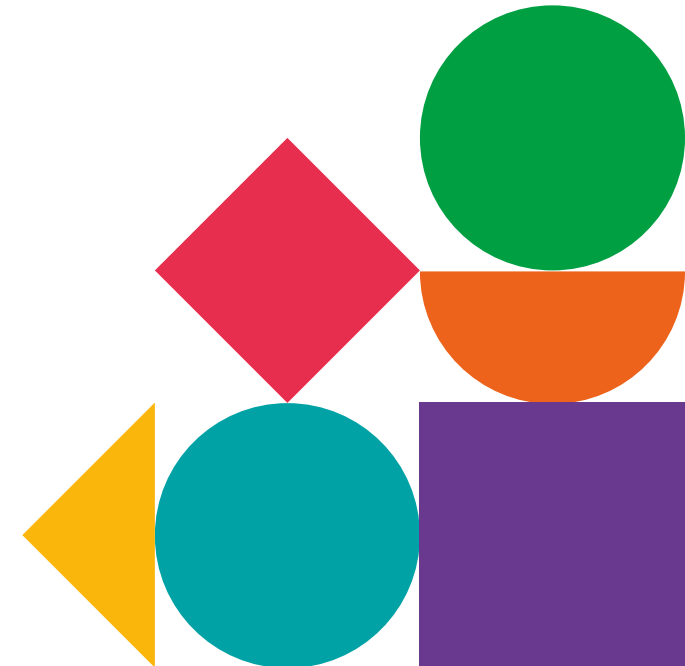
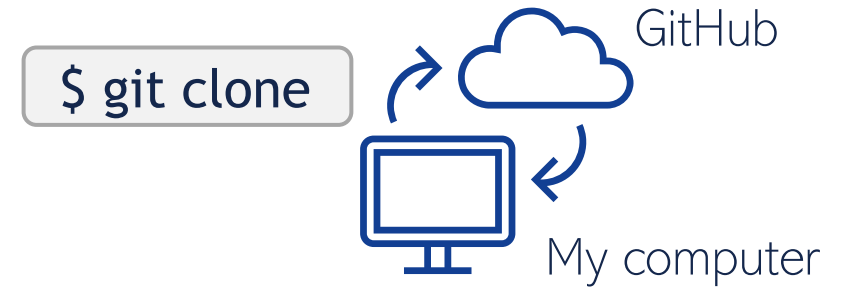
- Before starting
 - Check the status, add >>> commit >>> push
 - Is your working tree clean?
 - AVOID CONFLICTS FOR NOW!
 - Solving conflicts is the next topic.



ACTIVITY

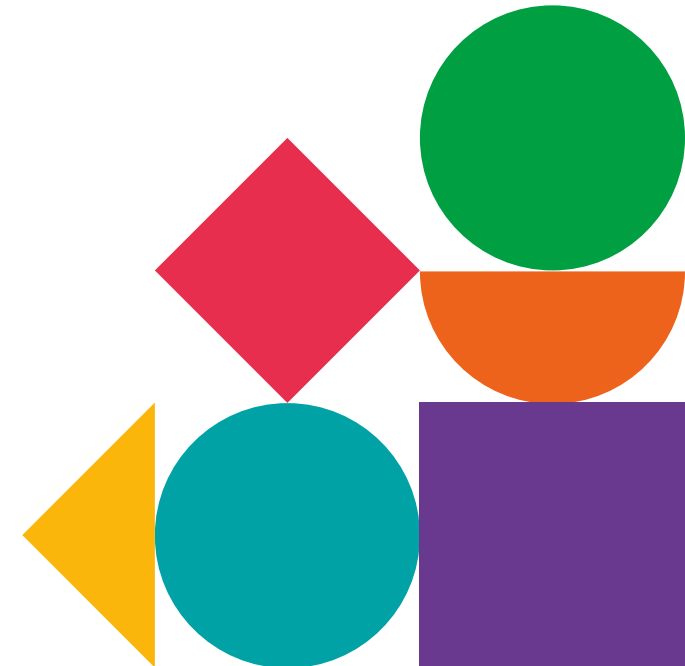
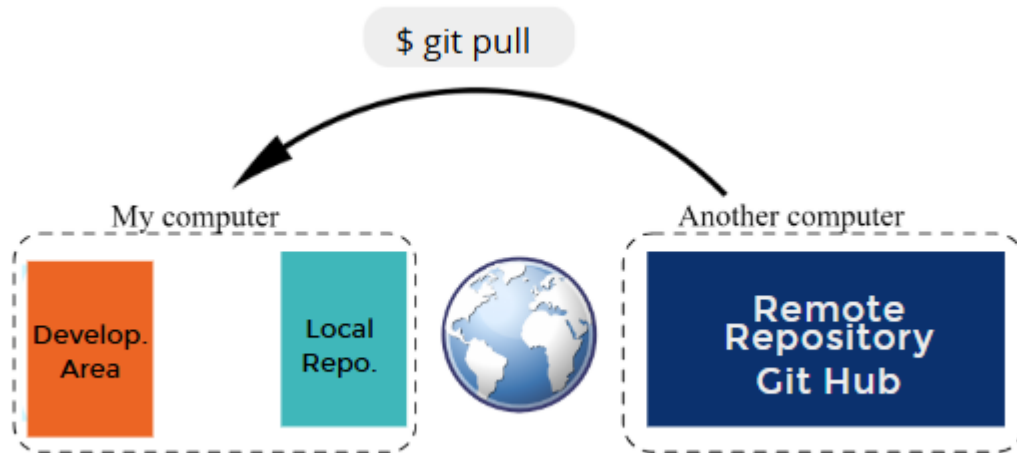
7. We will work together

- Help your partner
 - Clone their repository
 - Be careful to do outside your own project
 - Create a branch
 - Write what they need to do to start a collaboration
 - Write about how using branches is good for collaborations
- Commit and push



ACTIVITY

8. Bring the changes of your project to your local repo



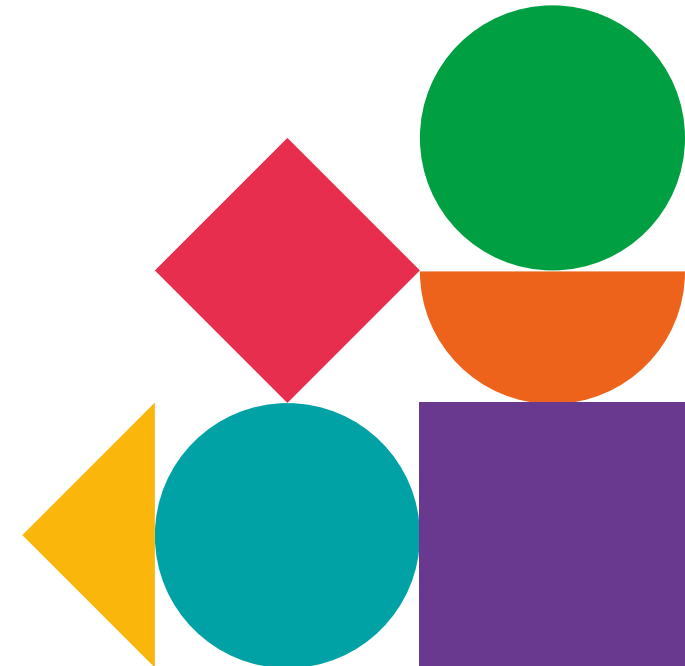
New branches and collaborations

```
user$ git branch -- list
```

- Check local branches

```
user$ git branch -a
```

- Check all branches
 - Local
 - Remote

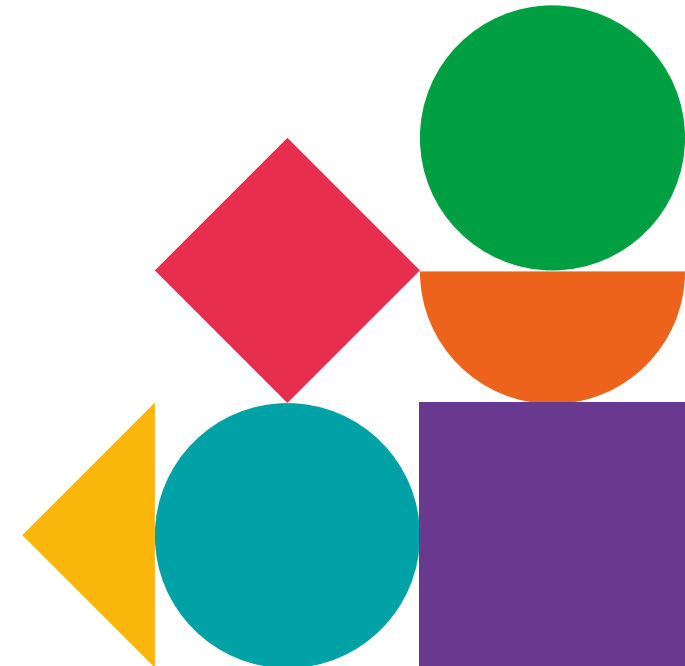


ACTIVITY

9. Go back to your project
 - Sync the content
 - Check the updates in the new branch your collaborator has created

```
user$ git branch - a
```

```
user$ git checkout <name>
```



QUIZ

 Kahoot!



Solving

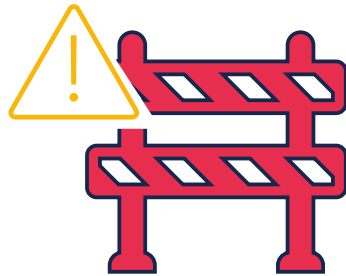


conflicts

ACTIVITY

17. Creating conflicts to solve them later

- Choose one project, work in the same file
- Write a message in the same line of the file
 - Commit and push



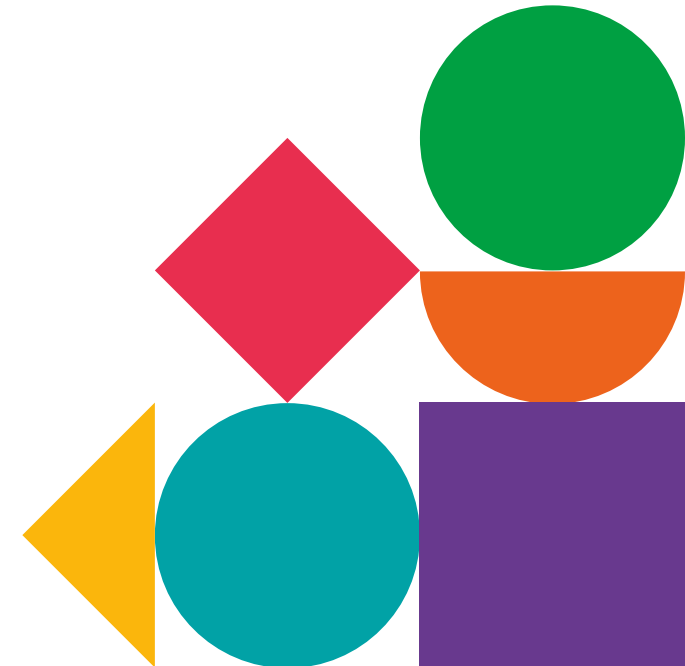
THE LAST ONE
WILL BE IN TROUBLE

conflict to solve

Auto-merging README.md

CONFLICT (content): Merge conflict in README.md

Automatic merge failed; fix conflicts and then commit the result.



With great power, comes great responsibility

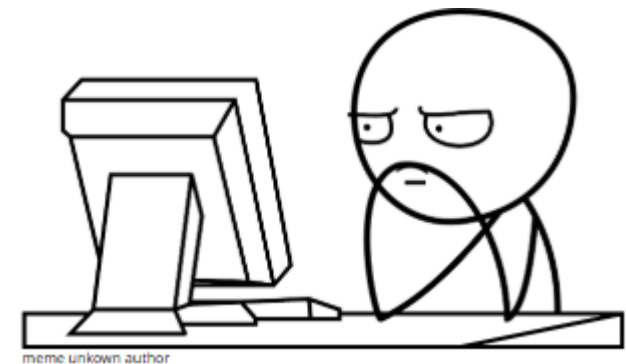
- Open the file where the conflict is found

```
<<<<<< HEAD
Text 1
=====
Text 2
>>>>>>> 587SD574
```

```
30
31 <<<<<< HEAD
32 I'm Scmander, and Pickett is my friend.
33 =====
34 I'm Pickett, a male Bowtruckle
35 >>>>>>> Bowtruckle
36
```

----- commit A

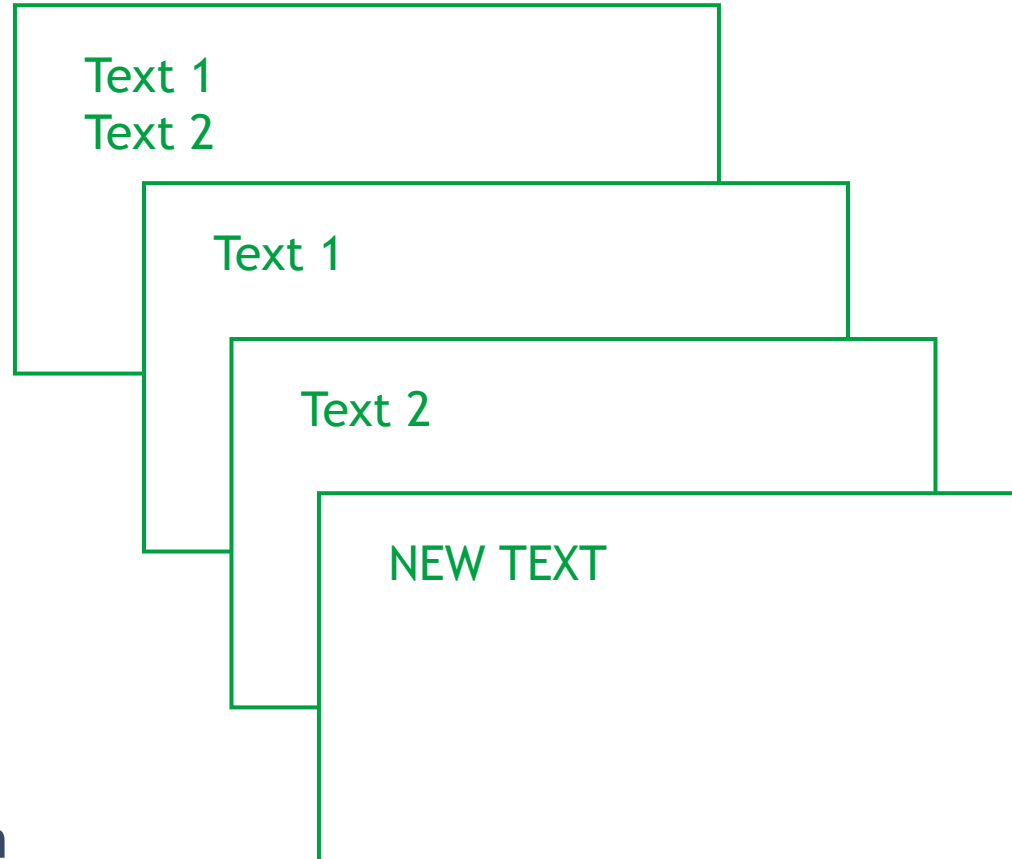
----- commit B



With great power, comes great responsibility

- Open the file where the conflict is found

```
<<<<<< HEAD
Text 1
=====
Text 2
>>>>>>> 587SD574
```



- You can change the text
 - Chose the best solution
 - Merge both solution
 - Create a completely new solution

Save



Commit

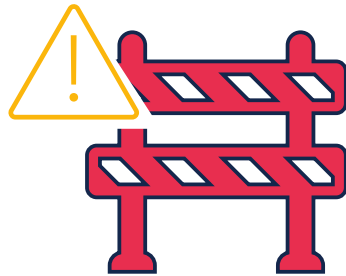


Push

ACTIVITY

18. Create a new conflict, your partner now has the chance to solve it

- Choose one project, work in the same file
- Write a message in the same line of the file
 - Commit and push

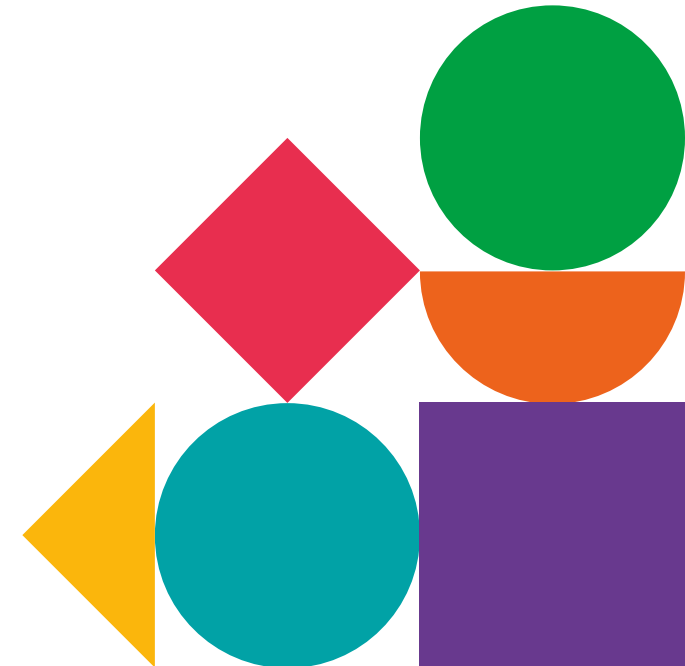


conflict to solve

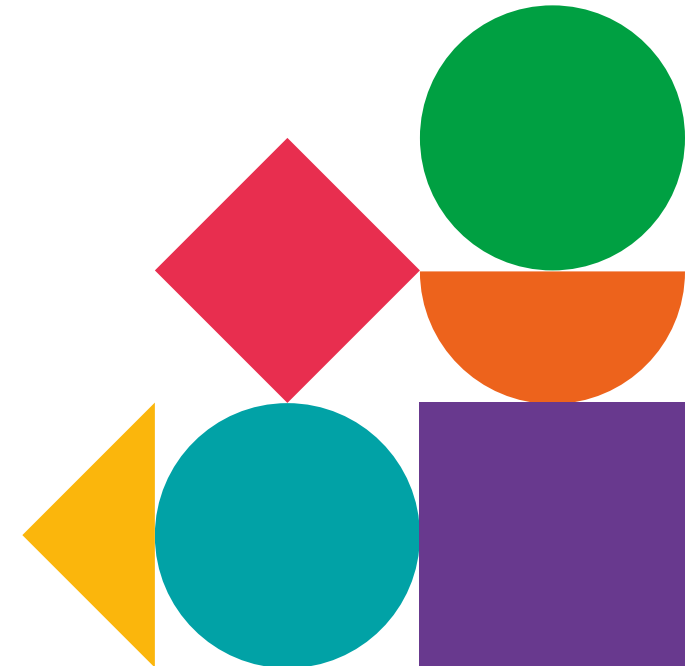
Auto-merging README.md

CONFLICT (content): Merge conflict in README.md

Automatic merge failed; fix conflicts and then commit the result.



QUESTIONS?



Merging

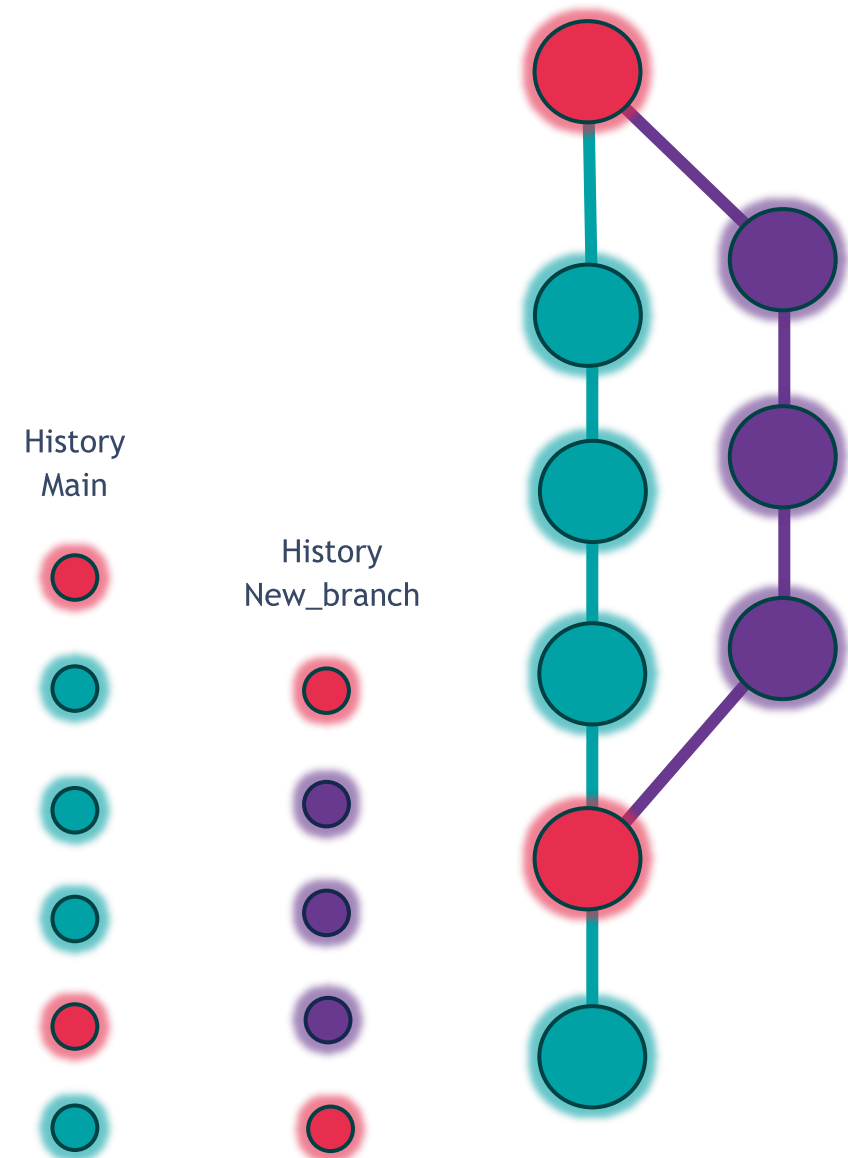
Branches come
together

Merging Branches

```
user$ git merge <branch_name>
```

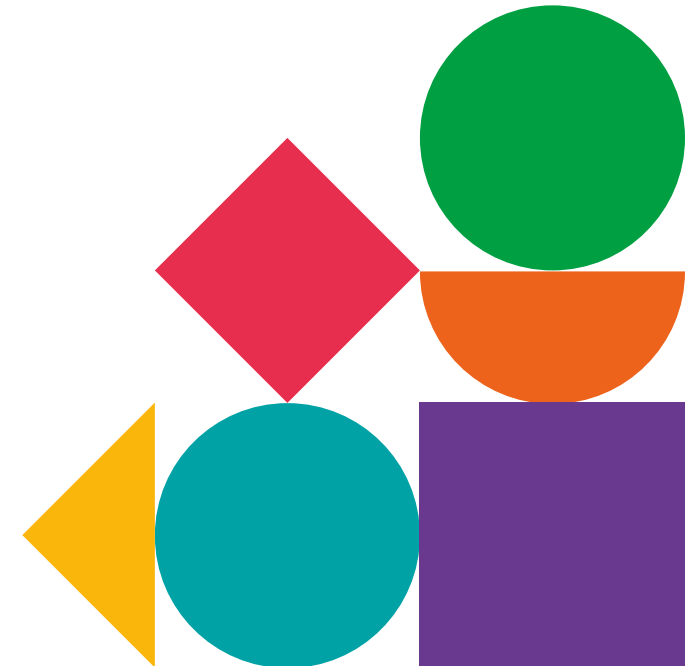
- Use cases
 1. Experiment worked out
 2. Finished collaboration
 3. Problem solved
- Too many branches
 - Clean up

```
user$ git branch -d <branch_name>
```



ACTIVITY

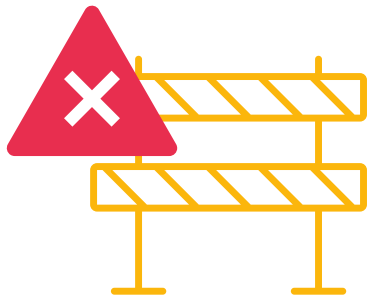
7. Create a new branch
 - Write about merging



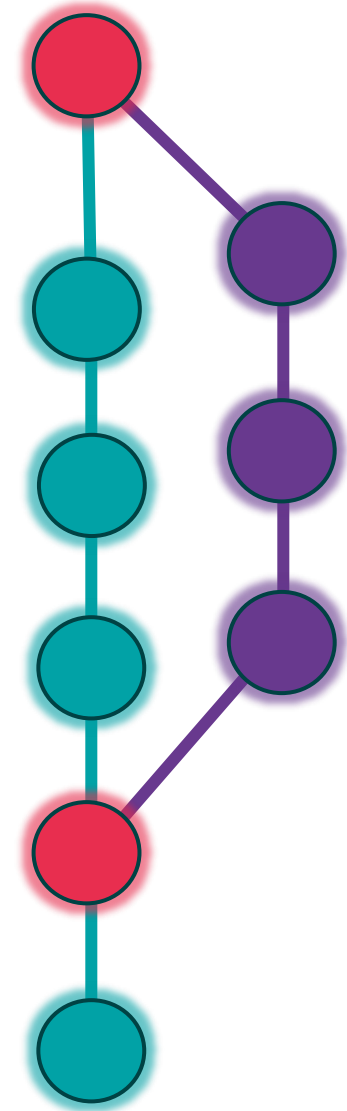
Merging Branches

```
user$ git merge <branch_name>
```

- Use cases
 1. Experiment worked out
 2. Finished collaboration
 3. Problem solved



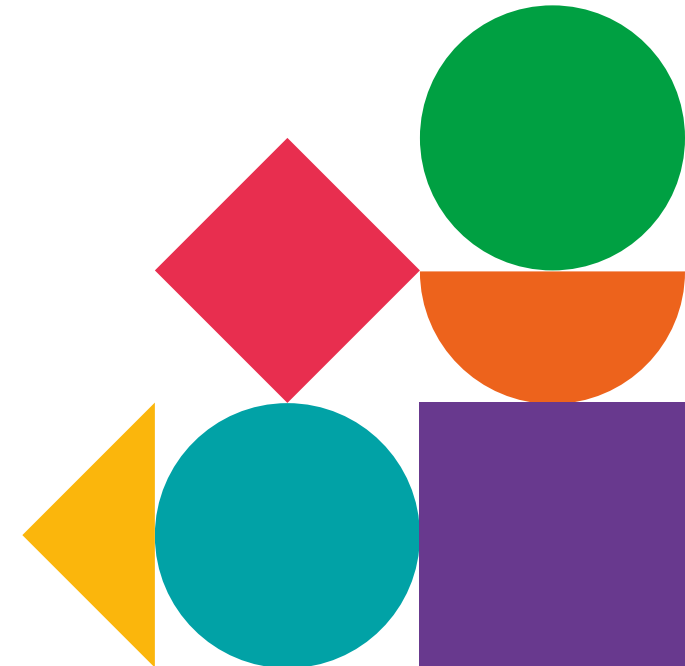
Challenge !!!
Solving conflicts



ACTIVITY

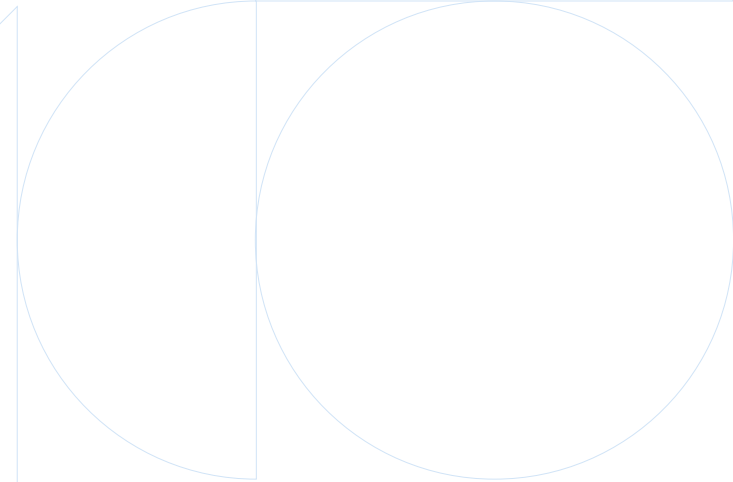
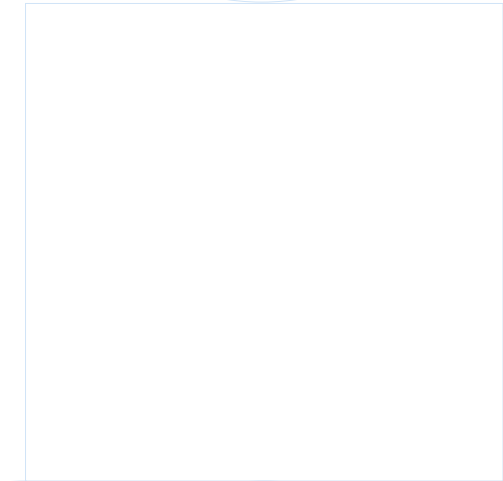
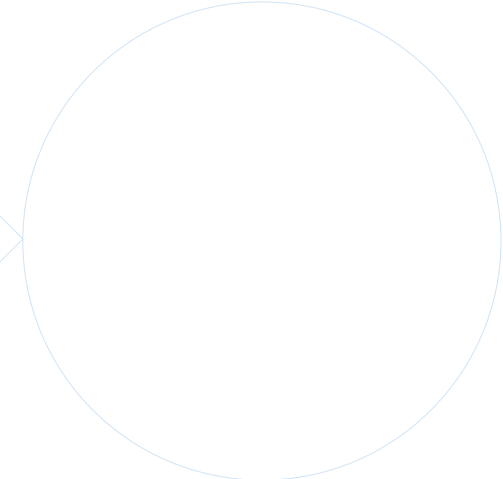
8. Merge one-by-one

- Merge all your branches with main
- Use conflict solving to keep only relevant information
- Think carefully of your commit messages
- Push your changes to GitHub



Change the past at your own risk

Checkout to an
old commit



Detached head

```
user$ git checkout <commit_ID>
```

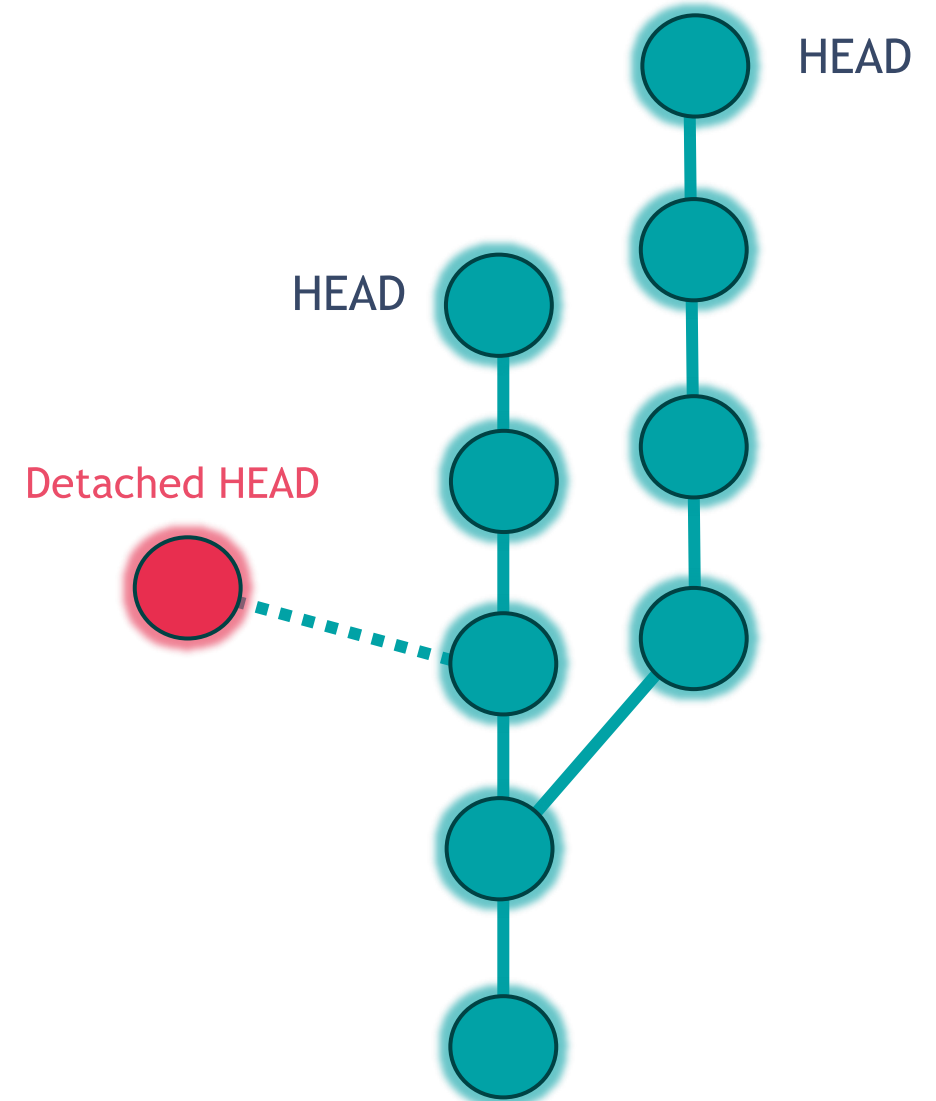
- You can't change the past
- You CAN create “an alternative future”
 - Be careful !!

You are in 'detached HEAD' state. You can look around, make experimental changes and commit them, and you can discard any commits you make in this state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may do so (now or later) by using `-c` with the switch command. Example:

```
git switch -c <new-branch-name>
```

```
user$ git switch -c <name_new_branch>
```



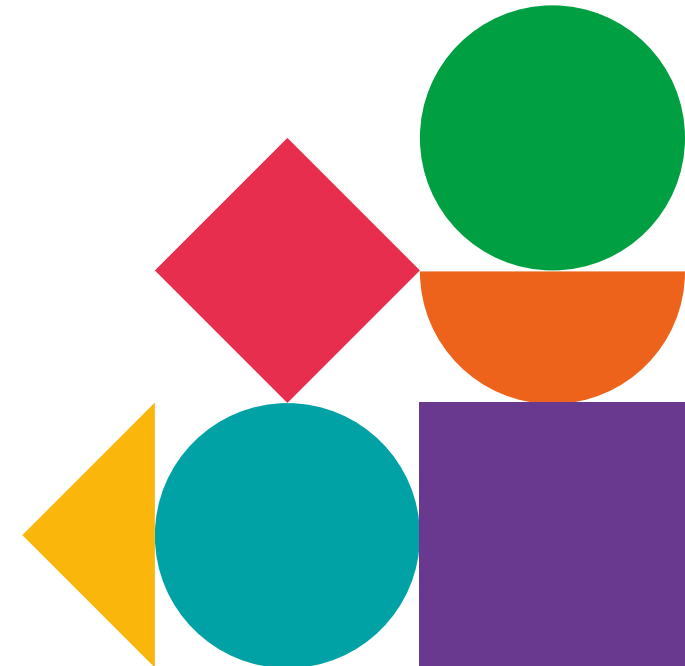
ACTIVITY

6. Changing the past

- Move to an old commit
- Add some text (2 steps)
 - Commit without switching
 - *Check in the list of branches*
 - Commit and do switch - c
 - *Check the list of branches*
- What happens?

```
user$ git checkout <commit_ID>
```

```
user$ git switch -c <name_new_branch>
```



Branches review

```
user$ git branch -- list
```

```
-a
```

```
<name>
```

List the local branches

List all branches, local & remote

Create a new branch

```
user$ git checkout <name / ID >
```

```
-b <name>
```

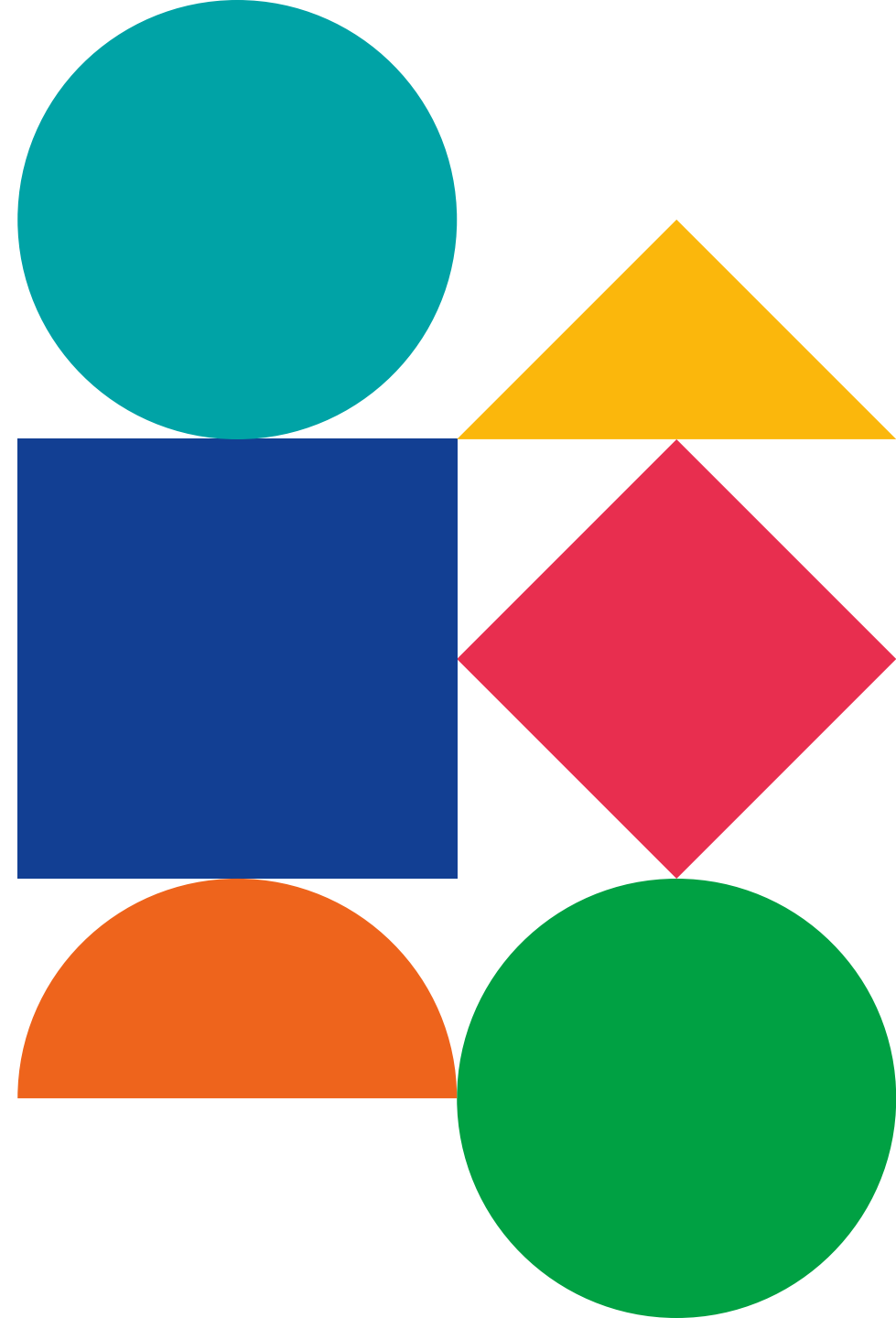
Go to branch

Create and go to new branch

```
user$ git switch -c <name_new_branch>
```

Transform a detached HEAD in a branch

REVIEW



Review

```
user$ git branch <branch_name>
```

- Creates an alternative timeline

```
user$ git checkout <branch_name>
```

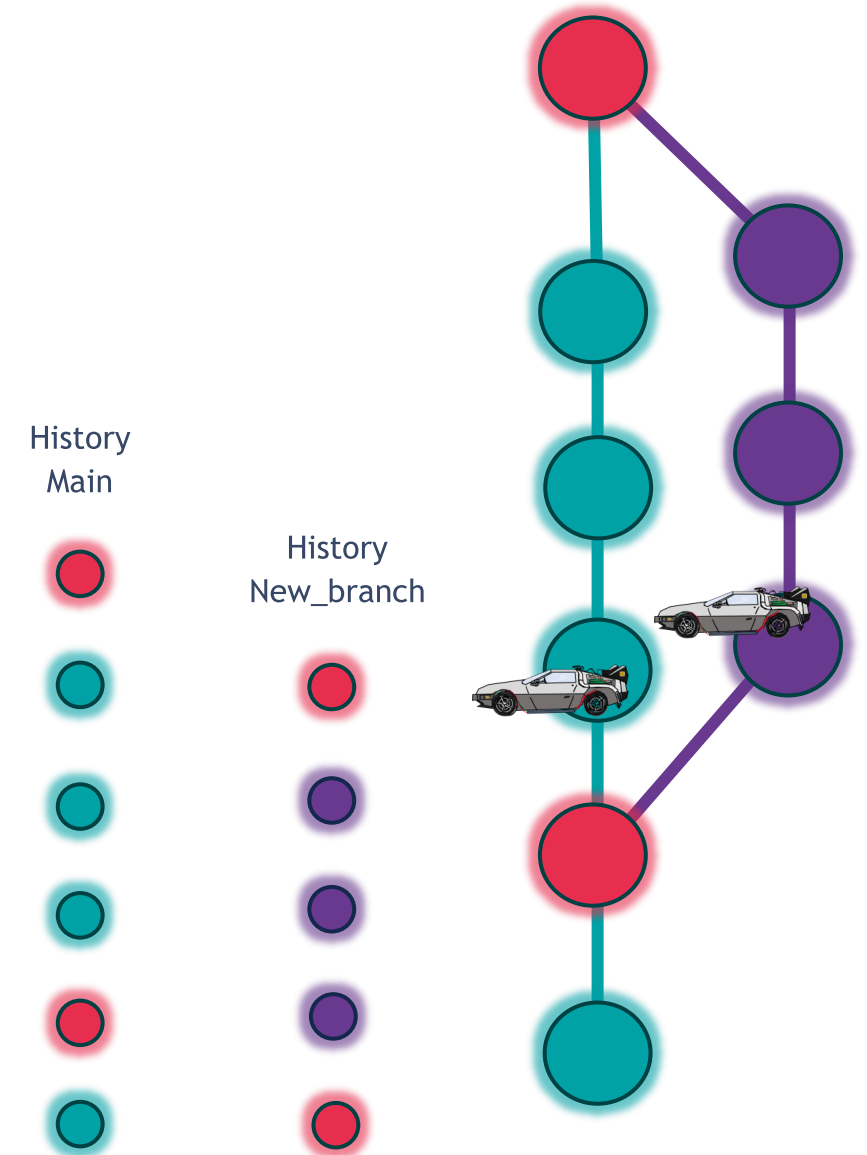
- Moves in the alternative timeline or old commit

```
user$ git checkout -b <branch_name>
```

- Creates and moves in the alternative timeline

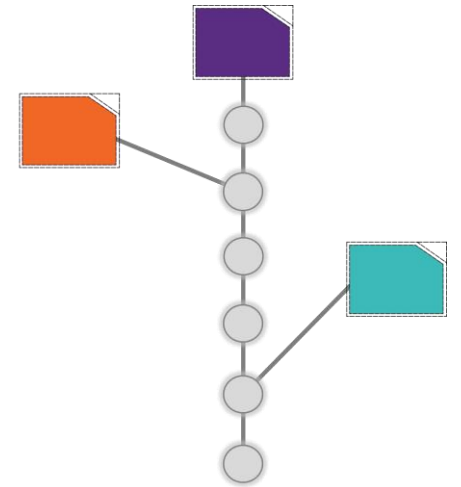
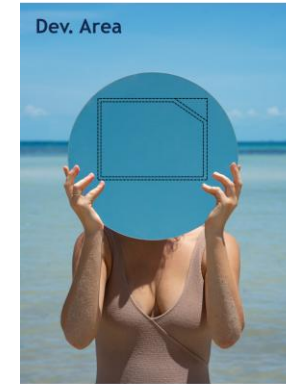
```
user$ git merge <branch_name>
```

- Merges two timelines in a shared commit point

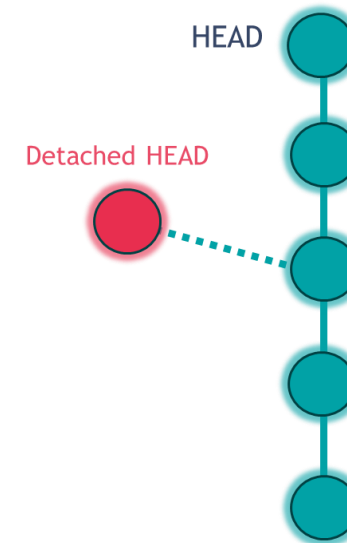


Review

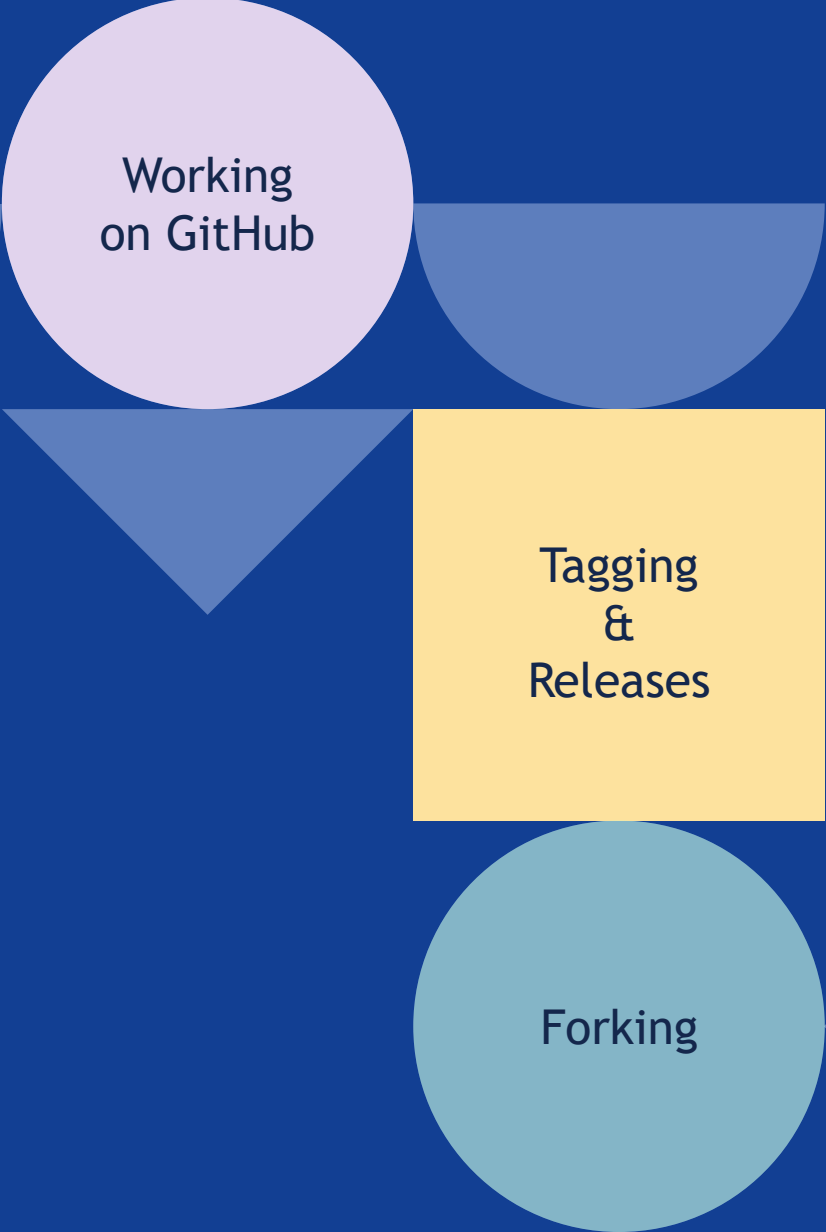
- The “mirror” effect
 - Folder Content == Branch you are working on



- Detached HEAD
 - Checking to old commit
 - A “floating” branch
 - Need to be connected



user\$ git switch -c <branch_name>

A diagram illustrating the relationship between three GitHub-related topics. At the top is a light purple circle labeled "Working on GitHub". Below it is a yellow square labeled "Tagging & Releases". At the bottom is a teal circle labeled "Forking". A blue diamond shape is positioned between the "Tagging & Releases" square and the "Forking" circle. The background is dark blue with various geometric shapes in lighter shades of blue.

Working
on GitHub

Tagging
&
Releases

Forking

3 topics
WHERE DO WE START?

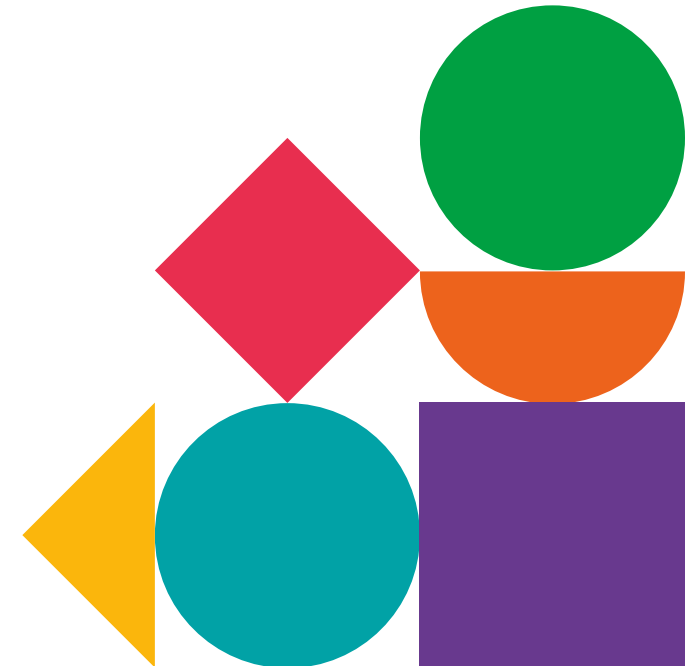


GitHub

Working
Online

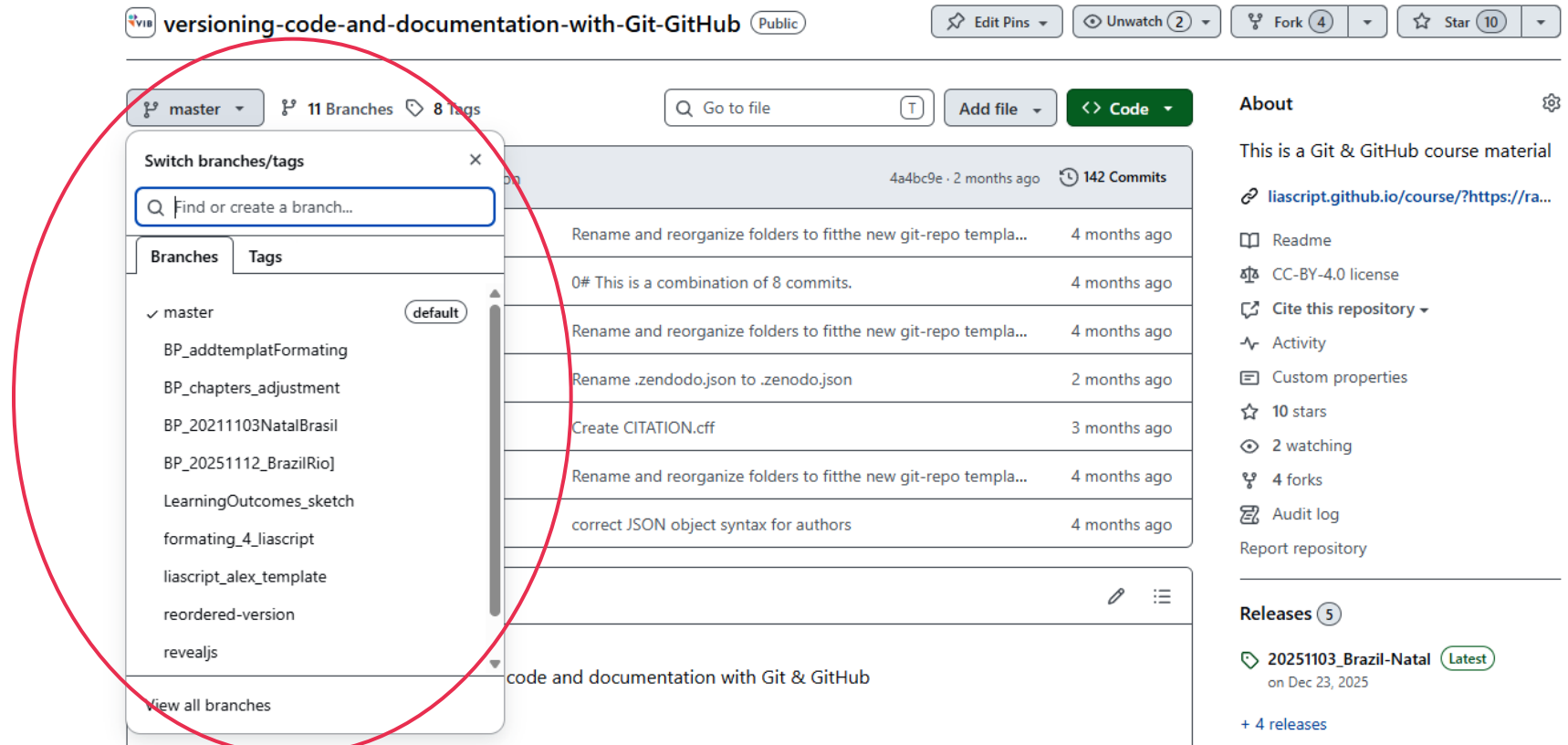
Basics on GitHub

- Check all branches
- Check all commits
- Editing
 - One-by-one
 - Several files
- Creating Issues
 - List of TODOs
 - List of bugs
- Tags and Releases



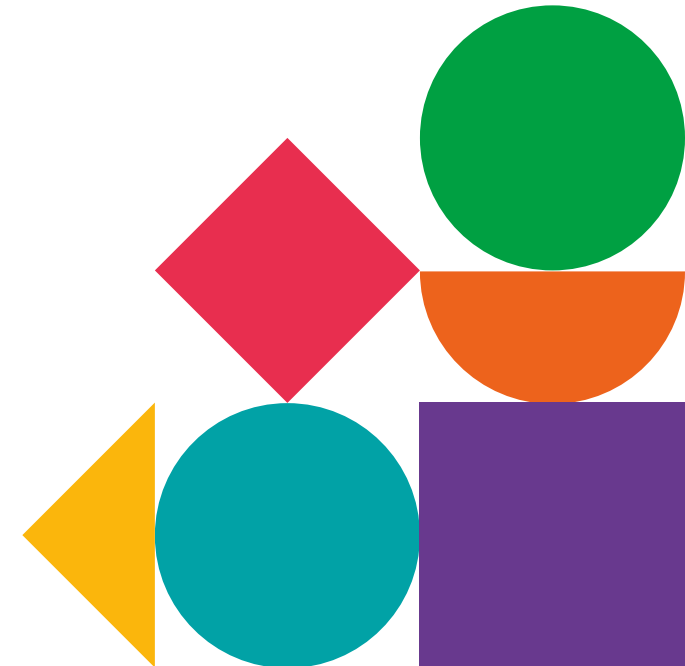
Basics on GitHub

- Check all branches
- Check all commits
- Editing
 - One-by-one
 - Several files
- Creating Issues
 - List of TODOs
 - List of bugs
- Tags and Releases



Basics on GitHub

- Check all branches
- Check all commits
- Editing
 - One-by-one
 - Several files
- Creating Issues
 - List of TODOs
 - List of bugs
- Tags and Releases



Basics on GitHub

- Check all branches
- Check all commits

 abotzki	Rename .zendodo.json to .zenodo.json	4a4bc9e · 2 months ago	🕒 142 Commits
data	Rename and reorganize folders to fit the new git-repo template...	4 months ago	
docs	0# This is a combination of 8 commits.	4 months ago	
exercises	Rename and reorganize folders to fit the new git-repo template...	4 months ago	
.zenodo.json	Rename .zendodo.json to .zenodo.json	2 months ago	
CITATION.cff	Create CITATION.cff	3 months ago	
LICENSE	Rename and reorganize folders to fit the new git-repo template...	4 months ago	

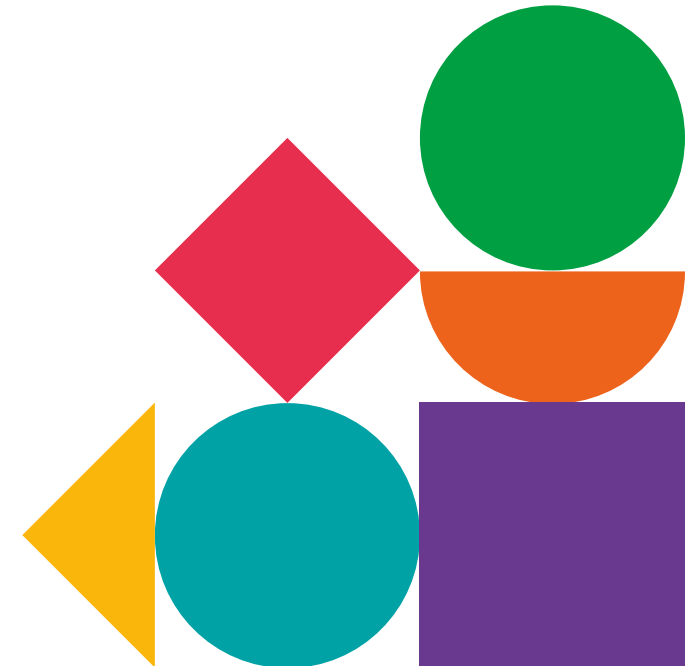
1 file changed +24 -23 lines changed

Search within code

151	- @orcid(Bruna Piareck)](http://orcid.org/00000-0001-5958-0669)	151	- @orcid(Bruna Piareck)](http://orcid.org/00000-0001-5958-0669)
152	- @orcid(James Collier)](http://orcid.org/0000-0002-0020-421X)	152	- @orcid(James Collier)](http://orcid.org/0000-0002-0020-421X)
153	- @orcid(Tuur Muyldermans)](http://orcid.org/0000-0002-3926-7293)	153	- @orcid(Tuur Muyldermans)](http://orcid.org/0000-0002-3926-7293)
154	- - @orcid(Alexander Botzki)](http://orcid.org/0000-0001-6691-4233)	154	+ - @orcid(Alexander Botzki)](http://orcid.org/0000-0001-6691-4233)
155		155	
156	## Citing this lesson	156	## Citing this lesson
157		157	
171		171	
172	Most things you need to get installed before hands, one or the other are optional tools to help with visualization and for somethings you can ask the trainer by mail or arriving 30 min before the class starts.	172	Most things you need to get installed before hands, one or the other are optional tools to help with visualization and for somethings you can ask the trainer by mail or arriving 30 min before the class starts.
173		173	
174	- [click here](https://liascript.github.io/course/?https://raw.githubusercontent.com/vib-tcp/introduction-github/refs/heads/0P_addtemplatFormatting/docs/tutorials/01_Get_ready_for_the_course.md#1)	174	+ [click here](https://liascript.github.io/course/?https://raw.githubusercontent.com/vib-tcp/introduction-github/refs/heads/master/docs/tutorials/01_Get_ready_for_the_course.md#1)
175		175	
176		176	

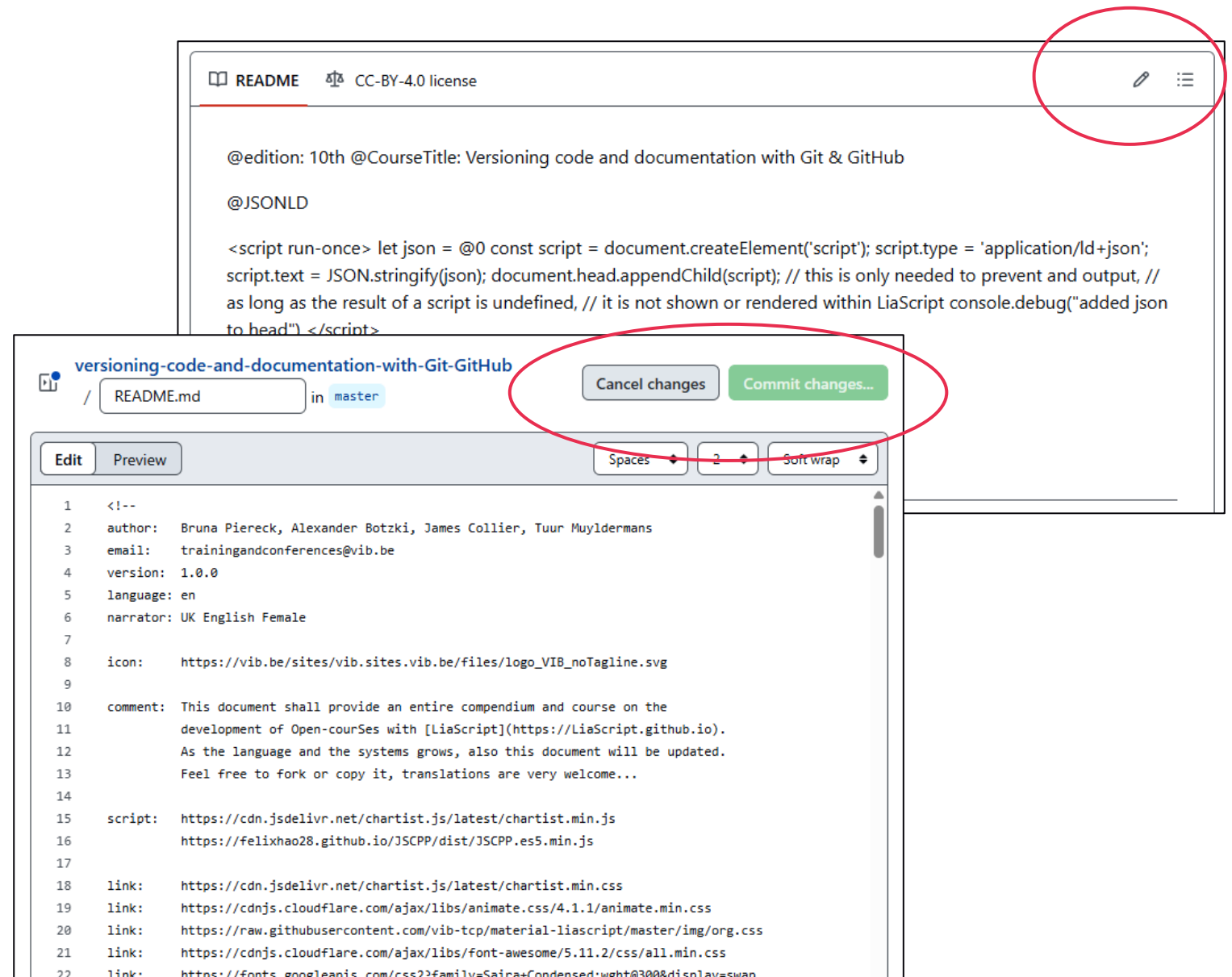
Basics on GitHub

- Check all branches
- Check all commits
- Editing
 - One-by-one
 - Several files
- Creating Issues
 - List of TODOs
 - List of bugs
- Tags and Releases



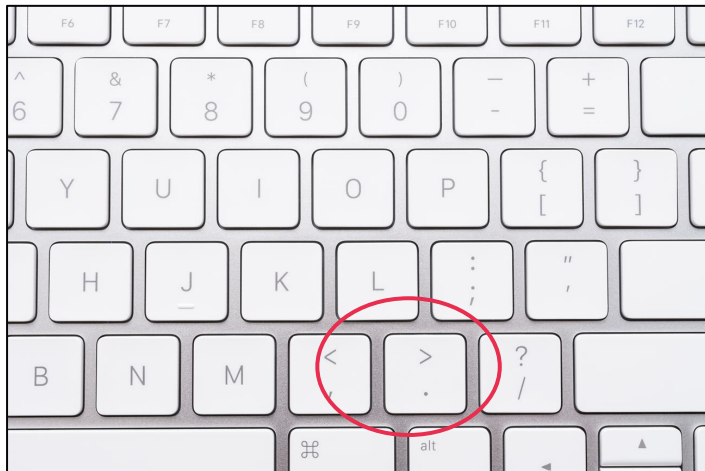
Basics on GitHub

- Check all branches
- Check all commits
- Editing
 - One-by-one
 - Several files
- Creating Issues
 - List of TODOs
 - List of bugs
- Tags and Releases



Basics on GitHub

- Check all branches
- Check all commits
- Editing
 - One-by-one
 - Several files



The top screenshot shows the GitHub repository page for 'versioning-code-and-documentation-with-Git-GitHub'. The 'Code' button is circled in red. The middle screenshot shows the 'README.md' file page, with the 'Edit file...' button in the toolbar circled in red. The bottom screenshot shows the 'No codespaces' message, with the 'Create codespace on master' button circled in red.

versioning-code-and-documentation-with-Git-GitHub Public

master 11 Branches 8 Tags

Go to file T Add file <> Code

Local Codespaces

Codespaces Your workspaces in the cloud + ...

No codespaces You don't have any codespaces with this repository checked out. Create codespace on master. Learn more about codespaces.

space usage for this repository is paid for by bpiereck.

t & GitHub

versioning-code-and-documentation-with-Git-GitHub

/ README.md

abotzki correct JSON object syntax for authors e0a0fa5 · 4 months ago

375 lines (247 loc) · 21.7 KB

Preview Code | Blame

Edit file... In place (E) Open with... github.dev (.) GitHub Desktop

@edition: 10th @CourseTitle: Versioning code and documentation with

@JSONLD

<script run-once> let json = @0 const script = document.createElement('application/ld+json'); script.text = JSON.stringify(json); document.head is only needed to prevent and output, // as long as the result of a script shown or rendered within LiaScript console.debug("added json to head") </script>

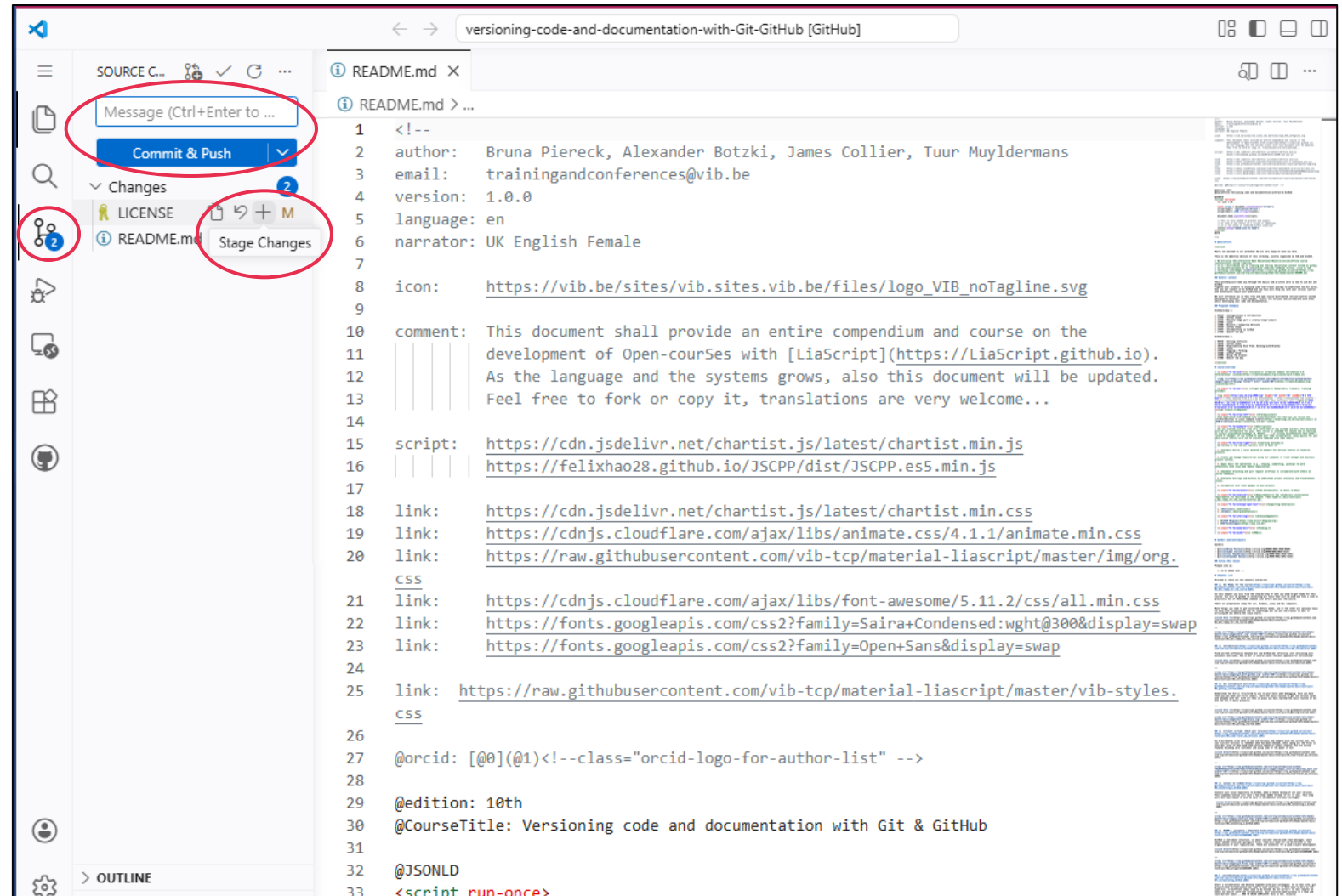
@end

-->

@CourseTitle

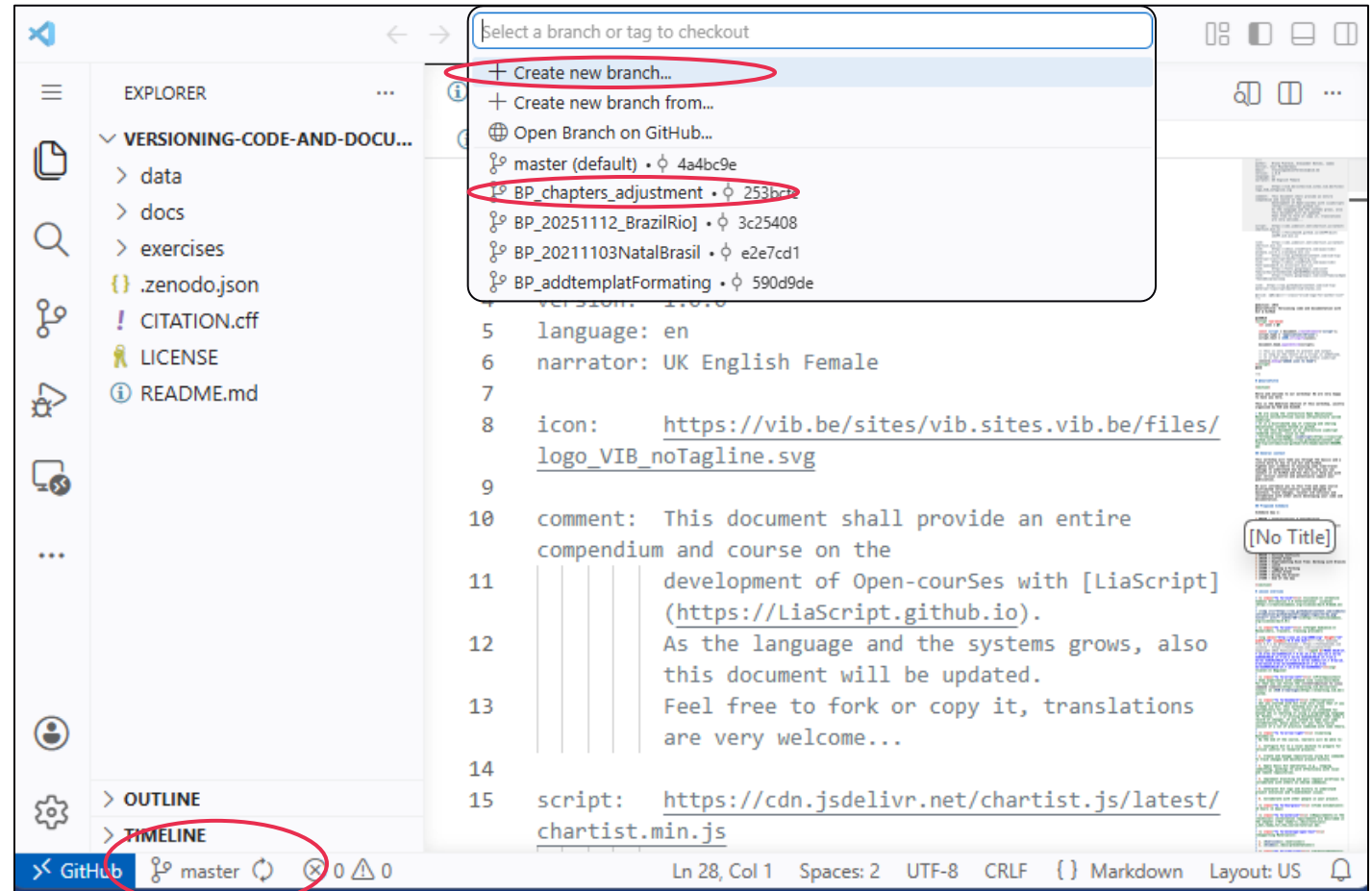
Basics on GitHub

- Check all branches
- Check all commits
- **Editing**
 - One-by-one
 - **Several files**
- Creating Issues
 - List of TODOs
 - List of bugs
- Tags and Releases



Basics on GitHub

- Check all branches
- Check all commits
- Editing
 - One-by-one
 - Several files
- Creating Issues
 - List of TODOs
 - List of bugs
- Tags and Releases



Branches

Basics on GitHub

- Check all branches
- Check all commits
- Editing
 - One-by-one
 - Several files
- Creating Issues
 - List of TODOs
 - List of bugs
- Tags and Releases

The screenshot shows the GitHub interface for creating a new issue. The 'Issues' tab is highlighted with a red circle. The 'Create new issue' dialog box is open, showing the following fields and options:

- Add a title ***: A text input field with the placeholder 'Title'.
- Add a description**: A text area with the placeholder 'Type your description here...'. It includes a rich text editor toolbar with options for bold, italic, link, and other formatting.
- Assignees**: A dropdown menu showing 'No one - [Assign yourself](#)'.
- Labels**: A dropdown menu showing 'No labels'.
- Type**: A dropdown menu showing 'No type'.
- Projects**: A dropdown menu showing 'No projects'.
- Milestone**: A dropdown menu showing 'No milestone'.

At the bottom of the dialog, there are buttons for 'Create more', 'Cancel', and 'Create'.

Basics on GitHub

- Check all branches
- Check all commits
- Editing
 - One-by-one
 - Several files
- Creating Issues
 - List of TODOs
 - List of bugs
- Tags and Releases

The screenshot shows a GitHub file view for the file `Git_QUIZ_intro2ndDay_En.md`. The interface includes a header with the repository name and a breadcrumb trail. Below the header, there are tabs for `Preview`, `Code`, and `Blame`. The `Code` tab is selected. The code content is displayed with line numbers. A context menu is open over line 10, which contains the following options: `Copy line`, `Copy permalink`, `View git blame`, `Reference in new issue`, `View file in GitHub.dev`, and `View file in different branch/tag`. The `Reference in new issue` option is highlighted. The code content includes a title, a mainfont, and a list of questions.

versioning-code-and-documentation-with-Git-GitHub / exercises
/ Git_QUIZ_intro2ndDay_En.md

bpiereck Rename and reorganize folders to fit the new git-repo templ... bf42468 · 4 months ago History

Preview Code Blame

1 ---
2 title: "Git & GitHub intro QUIZ for 2nd day English version"
3 mainfont: "Liberation Mono"
4 ---
5
6 # 1. What is True about Git / GitHub?
7
8 a. GitHub is software for versioning
9
10 b. Git is a website to keep backups of git projects
11
12 c. Git is a website to keep backups and collaborations
13
14
15
16
17
18
19
20
21
22
23 c. `git commit -m 'project description'`

Copy line
Copy permalink
View git blame
Reference in new issue
View file in GitHub.dev
View file in different branch/tag

Basics on GitHub

- Check all branches
- Check all commits
- Editing
 - One-by-one
 - Several files
- Creating Issues
 - List of TODOs
 - List of bugs
- Tags and Releases

Creating ISSUES:

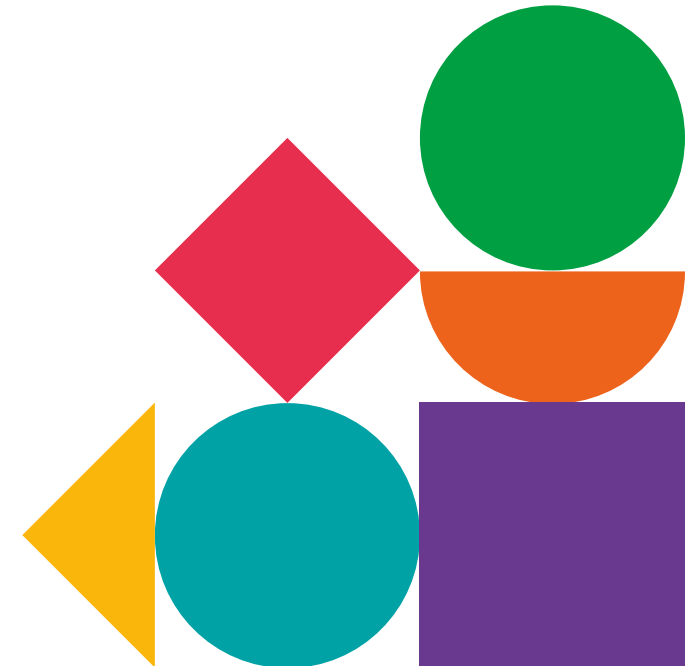
1. Descriptive title.
2. Explains the purpose of the issue
3. Include details that could help resolve the issue.

You can:

1. Create list of tasks
2. Create labels
3. Assign Issues to a collaborator
4. Exchange messages in a dedicated "chat"
5. Reference a line or paragraph

Basics on GitHub

- Check all branches
- Check all commits
- Editing
 - One-by-one
 - Several files
- Creating Issues
 - List of TODOs
 - List of bugs
- Tags and Releases



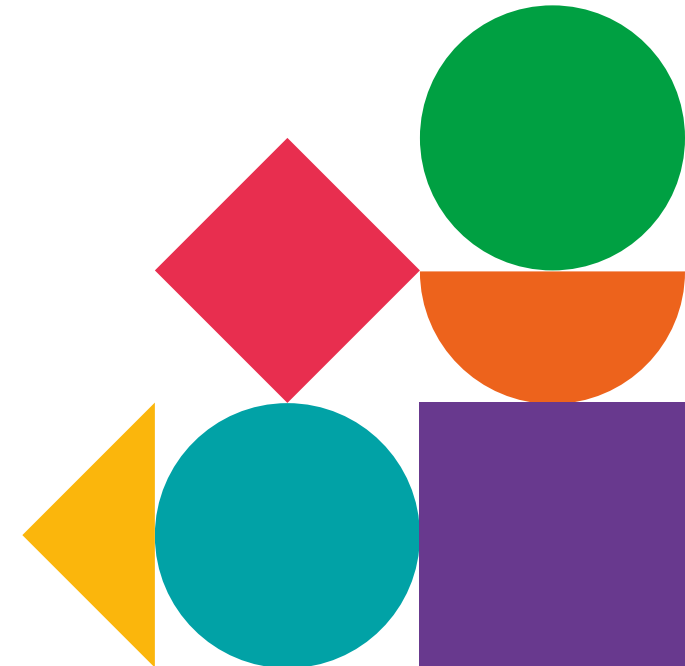
Tagging

Highlight
good versions

Tagging the commit

```
user$ git tag <name>
```

- Tag an important commit
 1. A good version
 2. A version for release
 3. A specific feature
 4. Experiment risk free
 5. Etc ..



Tagging the commit

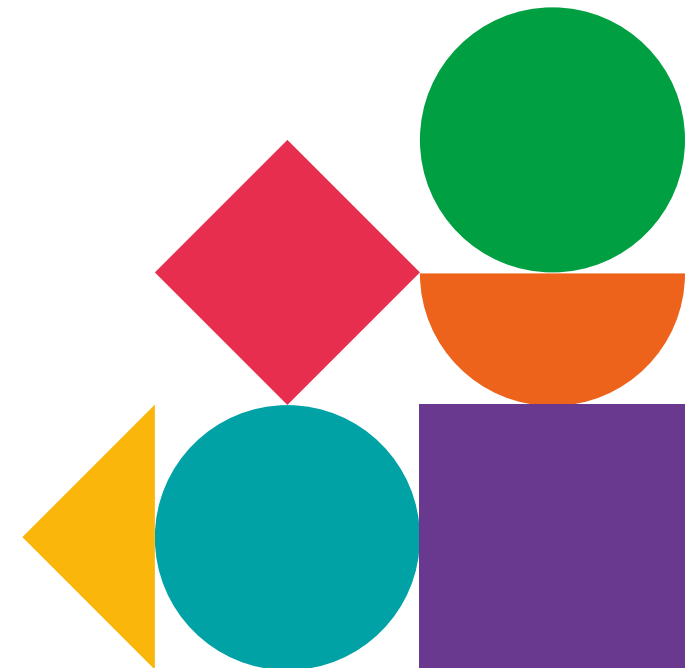
```
user$ git tag <name>
```

- Tag an important commit

1. A good version
2. A version for release
3. A specific feature
4. Experiment risk free
5. Etc ..

- Listing existing tags

```
user$ git tag  
user$ git tag -l  
user$ git tag --list
```



Tagging the commit

```
user$ git tag <name>
```

- Tag an important commit

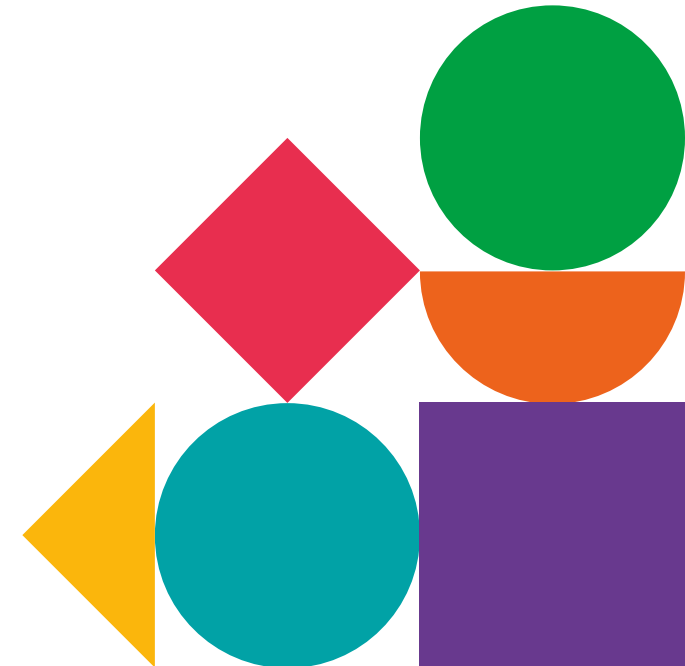
1. A good version
2. A version for release
3. A specific feature
4. Experiment risk free
5. Etc ..

- Listing existing tags

```
user$ git tag  
user$ git tag -l  
user$ git tag --list
```

- Tag an old commit

```
user$ git tag <name> <commit_ID>
```



ACTIVITY

9. Annotating

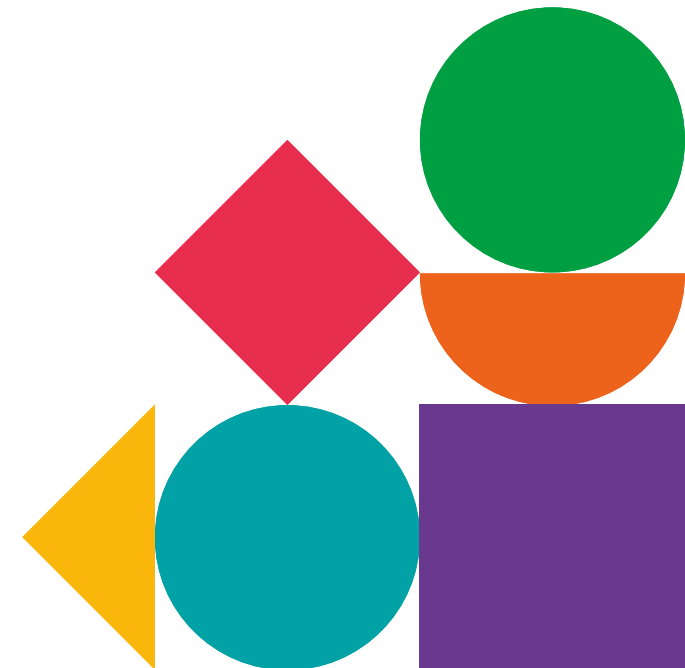
- Annotate in your documents about what is the tag used for
- What are the commands
- Commit

10. Creating tags

- Tag the last commit as “info_tag”
- Choose an old commit and create a tag for it
- List your tags

11. Challenge: Push your tags to GitHub

```
user$ git push --tags
```



Tags and GitHub releases

- Tag == commit “name”
- GitHub release

The screenshot shows the GitHub interface for the repository 'versioning-code-and-documentation-with-Git-GitHub'. At the top, the repository name is followed by 'Public' and buttons for 'Edit Pins', 'Unwatch (2)', 'Fork (4)', and 'Star (10)'. Below this, the navigation bar shows 'master' as the selected branch, '11 Branches', and '8 Tags' (which is circled in red). A search bar 'Go to file' and buttons for 'Add file' and 'Code' are also present. The main content area displays a list of files and folders with their commit messages and timestamps. The files listed are: 'data' (Rename and reorganize folders to fit the new git-repo template... 4 months ago), 'docs' (0# This is a combination of 8 commits. 4 months ago), 'exercises' (Rename and reorganize folders to fit the new git-repo template... 4 months ago), '.zenodo.json' (Rename .zendodo.json to .zenodo.json 2 months ago), 'CITATION.cff' (Create CITATION.cff 3 months ago), 'LICENSE' (Rename and reorganize folders to fit the new git-repo template... 4 months ago), and 'README.md' (correct JSON object syntax for authors 4 months ago). Below the file list, the 'README' tab is selected, showing the repository's description: '@edition: 10th @CourseTitle: Versioning code and documentation with Git & GitHub' and '@JSONLD'. On the right side, the 'About' section provides information about the repository, including a link to 'lisscript.github.io/course/?https://ra...', a 'Readme' link, 'CC-BY-4.0 license', 'Cite this repository', 'Activity', 'Custom properties', '10 stars', '2 watching', '4 forks', and 'Audit log'. At the bottom right, the 'Releases (5)' section is circled in red, showing a list of releases. The latest release is '20251103_Brazil-Natal' on Dec 23, 2025, marked as 'Latest'. There are also links for '+ 4 releases'.

File/Folder	Commit Message	Timestamp
data	Rename and reorganize folders to fit the new git-repo template...	4 months ago
docs	0# This is a combination of 8 commits.	4 months ago
exercises	Rename and reorganize folders to fit the new git-repo template...	4 months ago
.zenodo.json	Rename .zendodo.json to .zenodo.json	2 months ago
CITATION.cff	Create CITATION.cff	3 months ago
LICENSE	Rename and reorganize folders to fit the new git-repo template...	4 months ago
README.md	correct JSON object syntax for authors	4 months ago

Releases (5)

- 20251103_Brazil-Natal (Latest) on Dec 23, 2025
- + 4 releases

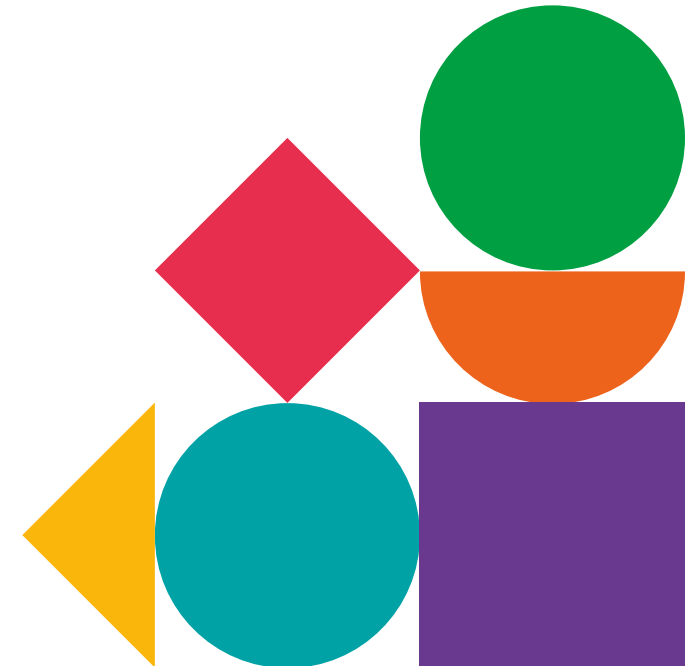
Tags and GitHub releases

- Tag == commit “name”
- GitHub release
 - Are based on tags
 - Allow for notes explanations
 - Can be sync to Zenodo for DOI

The screenshot displays a GitHub repository interface. At the top, the repository name 'versioning-code-and-documentation-with-Git-GitHub' is visible, along with navigation links like 'Edit Pins', 'Unwatch', 'Fork', and 'Star'. Below this, the 'master' branch is selected, and the 'Tags' tab is highlighted with a red circle. The repository has 11 branches and 8 tags. A commit history table shows a recent commit by 'bpierack' titled '20251112_BrazilRio' with a commit hash of '3c25408'. A 'Draft a new release' button is circled in red. A 'New release' modal is open, showing fields for 'Tag: Select tag', 'Target: master', 'Release title', and 'Release notes'. The modal also includes a 'Generate release notes' button and a text area for describing the release. The repository's README is visible in the background, showing a file tree with 'data', 'docs', 'exercises', '.zenodo.json', 'CITATION.cff', 'LICENSE', and 'README.md'.

ACTIVITY

12. Create a Release o GitHub

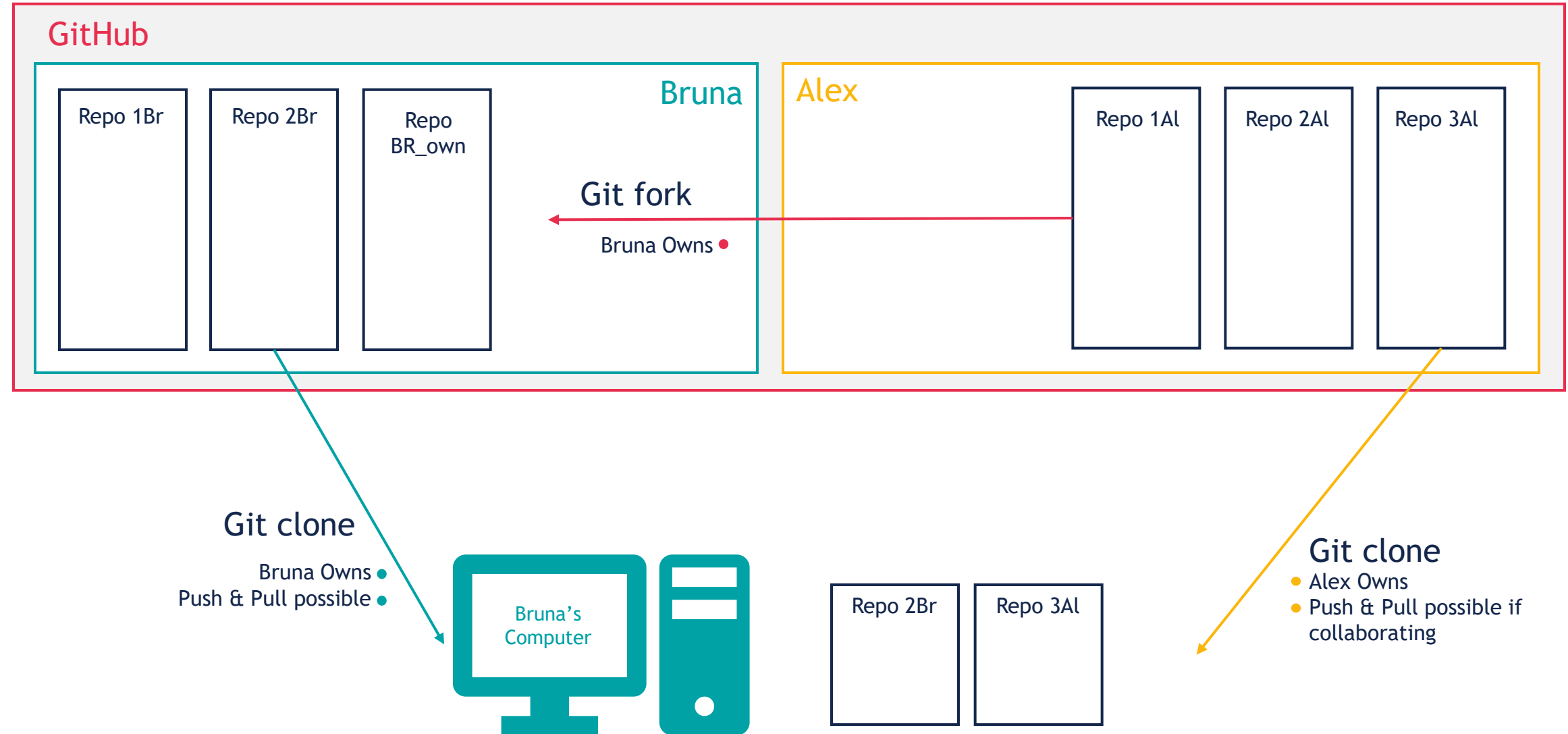


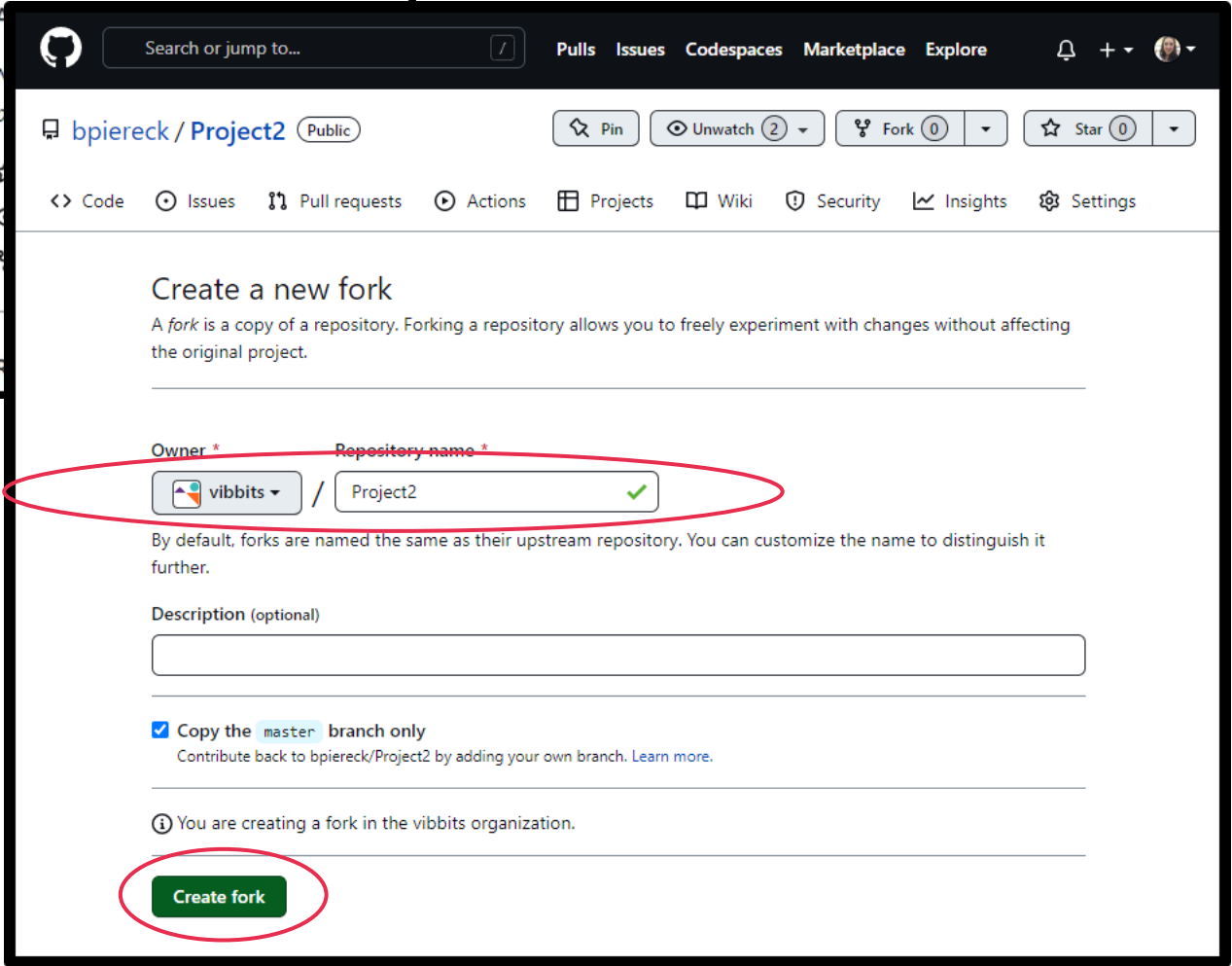
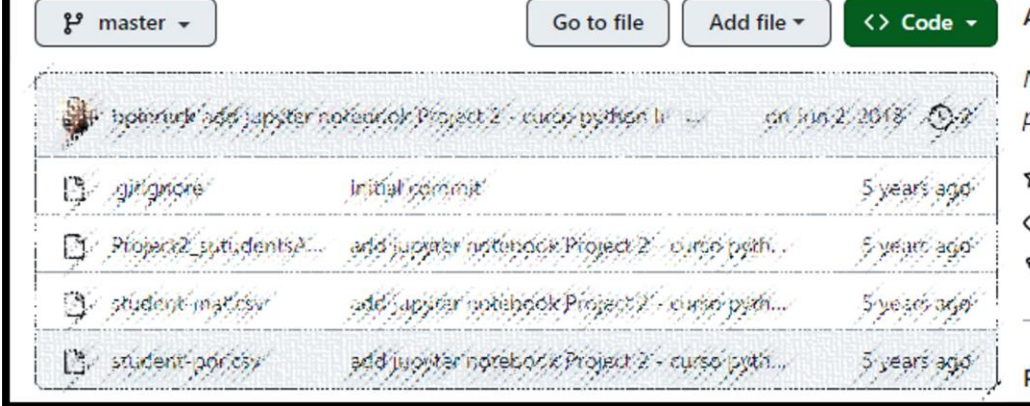
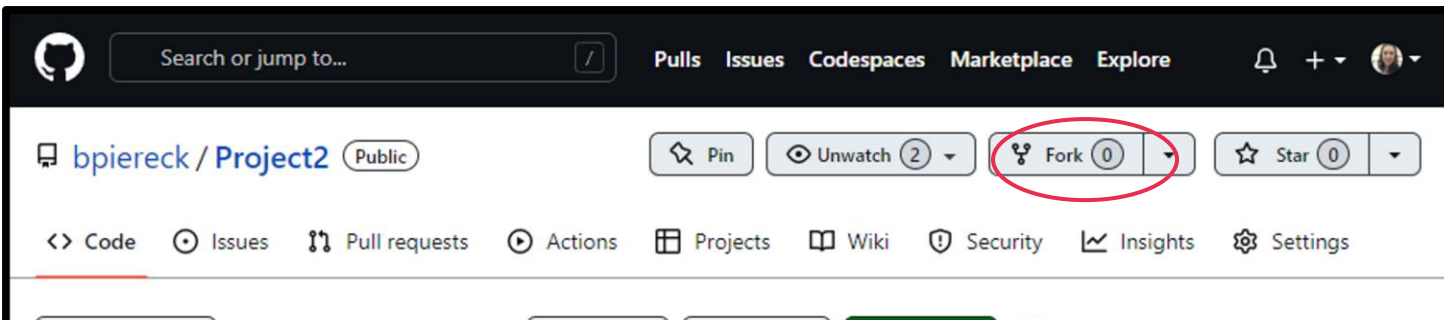
Forking



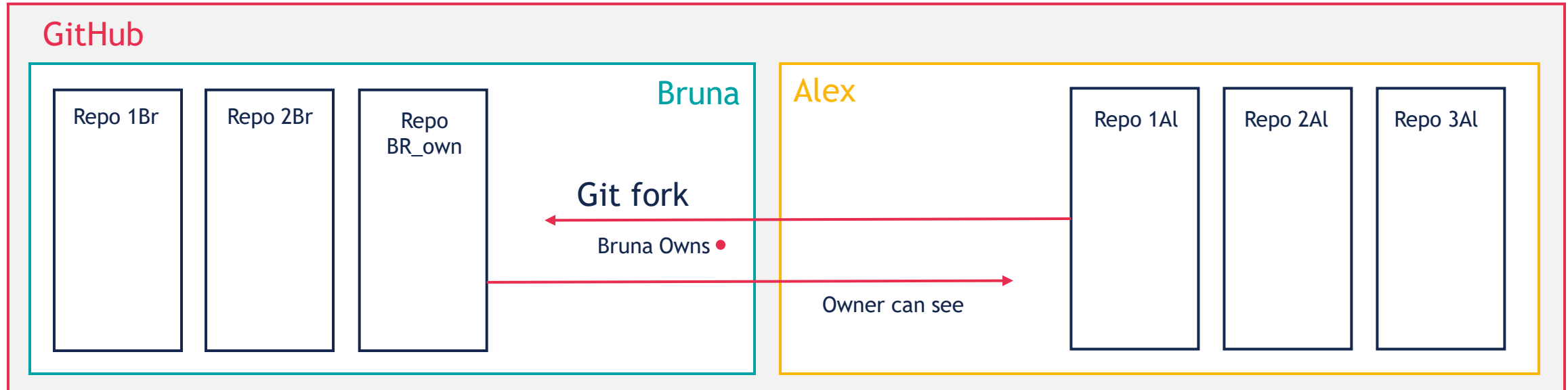
Copping
repositories

Fork vs Clone





Fork vs Clone



- Still connected
- Original creator can see it
- Changes can be incorporated
 - Both directions
 - Pull request

You are ready

In case of fire



1. git commit



2. git push

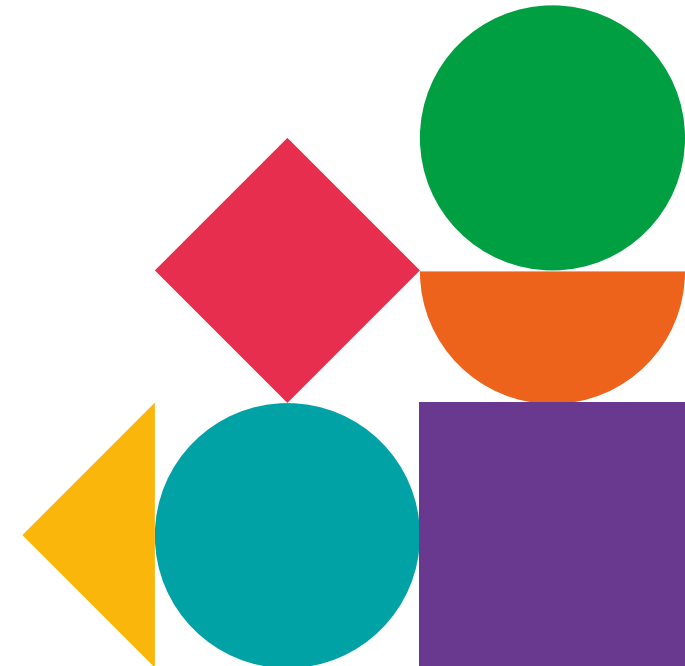


3. leave building

ACTIVITY

13. Fork race

- Wait for instructions to be a fair race!
- Add your name to the last line of the README file
- Commit (and wait)
- When instructed ask for a pull requested
 - A pull request must be evaluated and accepted





Bruna Piereck

Bruna.piereckmoura@vib.be

